

NHS_Exploration_Notebook

June 16, 2026

1 Business Context

1.0.1 The NHS wants to know why patients miss their scheduled appointments with their general practitioners (GPs). This objective is desired because missed appointments incur significant financial strains on the NHS, therefore being able to mitigate this occurrence is a high priority. Rather than imposing financial penalties on patients who miss their appointments, the NHS wants to embrace a data-informed approach. In lieu of this goal the NHS wants to know:

1.0.2 1) Whether there has been adequate staffing and capacity across their networks.

1.0.3 2) How their resources are actually being utilized.

```
[825]: #Import Packages
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import statsmodels.api as sm
import statsmodels.formula.api as smf
import warnings
warnings.filterwarnings("ignore")
```

```
[826]: #Import the data and analyzed in this order: (Commented out to avoid re-import)
AD = pd.read_csv('actual_duration.csv')
NC = pd.read_excel('national_categories.xlsx')
AR = pd.read_csv('appointments_regional.csv')
TW = pd.read_csv('tweets.csv')
```

```
[827]: #Set decimal settings to 4 decimals
pd.set_option("display.float_format", "{:.4f}".format)
```

2 Actual Duration Analysis

```
[828]: # View the df shape
AD.shape
```

```
[828]: (137793, 8)
```

```
[829]: #Mostly categorical data, only count of appts is an integer
print(AD.dtypes)
```

```
sub_icb_location_code    object
sub_icb_location_ons_code object
sub_icb_location_name    object
icb_ons_code             object
region_ons_code          object
appointment_date         object
actual_duration          object
count_of_appointments    int64
dtype: object
```

```
[830]: #No Null values observed
AD.isnull().sum()
```

```
[830]: sub_icb_location_code    0
sub_icb_location_ons_code    0
sub_icb_location_name       0
icb_ons_code                0
region_ons_code             0
appointment_date            0
actual_duration             0
count_of_appointments       0
dtype: int64
```

```
[831]: #No duplicates rows
print("Duplicates:",AD.duplicated().sum())
```

```
Duplicates: 0
```

```
[832]: #View top of dataset
#However there are "Unknown / Data Quality" values
AD.head()
```

```
[832]:  sub_icb_location_code  sub_icb_location_ons_code  \
0                00L          E38000130
1                00L          E38000130
2                00L          E38000130
3                00L          E38000130
```

4

00L

E38000130

	sub_icb_location_name	icb_ons_code	region_ons_code	\
0	NHS North East and North Cumbria ICB - 00L	E54000050	E40000012	
1	NHS North East and North Cumbria ICB - 00L	E54000050	E40000012	
2	NHS North East and North Cumbria ICB - 00L	E54000050	E40000012	
3	NHS North East and North Cumbria ICB - 00L	E54000050	E40000012	
4	NHS North East and North Cumbria ICB - 00L	E54000050	E40000012	

	appointment_date	actual_duration	count_of_appointments
0	01-Dec-21	31-60 Minutes	364
1	01-Dec-21	21-30 Minutes	619
2	01-Dec-21	6-10 Minutes	1698
3	01-Dec-21	Unknown / Data Quality	1277
4	01-Dec-21	16-20 Minutes	730

```
[833]: #View bottom of dataset
```

```
AD.tail()
```

```
[833]: sub_icb_location_code sub_icb_location_ons_code \
137788 X2C4Y E38000254
137789 X2C4Y E38000254
137790 X2C4Y E38000254
137791 X2C4Y E38000254
137792 X2C4Y E38000254
```

	sub_icb_location_name	icb_ons_code	region_ons_code	\
137788	NHS West Yorkshire ICB - X2C4Y	E54000054	E40000012	
137789	NHS West Yorkshire ICB - X2C4Y	E54000054	E40000012	
137790	NHS West Yorkshire ICB - X2C4Y	E54000054	E40000012	
137791	NHS West Yorkshire ICB - X2C4Y	E54000054	E40000012	
137792	NHS West Yorkshire ICB - X2C4Y	E54000054	E40000012	

	appointment_date	actual_duration	count_of_appointments
137788	30-Jun-22	31-60 Minutes	430
137789	30-Jun-22	21-30 Minutes	751
137790	30-Jun-22	16-20 Minutes	921
137791	30-Jun-22	11-15 Minutes	1439
137792	30-Jun-22	1-5 Minutes	1739

```
[834]: #View description of numerical data
```

```
AD.describe()
```

```
[834]: count_of_appointments
count      137793.0000
mean       1219.0800
std        1546.9030
```

min	1.0000
25%	194.0000
50%	696.0000
75%	1621.0000
max	15400.0000

```
[835]: #Rename columns to easier conventions
AD = AD.rename(columns={
    'sub_icb_location_code': 'ICB_Code',
    'sub_icb_location_ons_code': 'ONS_Code',
    'sub_icb_location_name': 'ICB_Name',
    'icb_ons_code': 'ICB_ONS_Code',
    'region_ons_code': 'ONS_R_Code',
    'appointment_date': 'Appt_Date',
    'actual_duration': 'Duration',
    'count_of_appointments': 'Appts',
})

print(AD)
```

	ICB_Code	ONS_Code	ICB_Name \
0	00L	E38000130	NHS North East and North Cumbria ICB - 00L
1	00L	E38000130	NHS North East and North Cumbria ICB - 00L
2	00L	E38000130	NHS North East and North Cumbria ICB - 00L
3	00L	E38000130	NHS North East and North Cumbria ICB - 00L
4	00L	E38000130	NHS North East and North Cumbria ICB - 00L
...	...	...	...
137788	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137789	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137790	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137791	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137792	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y

	ICB_ONS_Code	ONS_R_Code	Appt_Date	Duration	Appts
0	E54000050	E40000012	01-Dec-21	31-60 Minutes	364
1	E54000050	E40000012	01-Dec-21	21-30 Minutes	619
2	E54000050	E40000012	01-Dec-21	6-10 Minutes	1698
3	E54000050	E40000012	01-Dec-21	Unknown / Data Quality	1277
4	E54000050	E40000012	01-Dec-21	16-20 Minutes	730
...	...	...	...	...	...
137788	E54000054	E40000012	30-Jun-22	31-60 Minutes	430
137789	E54000054	E40000012	30-Jun-22	21-30 Minutes	751
137790	E54000054	E40000012	30-Jun-22	16-20 Minutes	921
137791	E54000054	E40000012	30-Jun-22	11-15 Minutes	1439
137792	E54000054	E40000012	30-Jun-22	1-5 Minutes	1739

[137793 rows x 8 columns]

```
[836]: #Make sure column names have changed
print(AD.columns)
```

```
Index(['ICB_Code', 'ONS_Code', 'ICB_Name', 'ICB_ONS_Code', 'ONS_R_Code',
      'Appt_Date', 'Duration', 'Appts'],
      dtype='object')
```

```
[837]: #See top 5 appointments
AD.sort_values(by="Appts", ascending=False).head()
```

```
[837]:      ICB_Code  ONS_Code      ICB_Name  ICB_ONS_Code  \
125518    A3A8R  E38000255  NHS North East London ICB - A3A8R    E54000029
125812    A3A8R  E38000255  NHS North East London ICB - A3A8R    E54000029
125861    A3A8R  E38000255  NHS North East London ICB - A3A8R    E54000029
126400    A3A8R  E38000255  NHS North East London ICB - A3A8R    E54000029
125616    A3A8R  E38000255  NHS North East London ICB - A3A8R    E54000029

      ONS_R_Code  Appt_Date      Duration  Appts
125518  E40000003  17-Jan-22  Unknown / Data Quality  15400
125812  E40000003  28-Feb-22  Unknown / Data Quality  15081
125861  E40000003  07-Mar-22  Unknown / Data Quality  14767
126400  E40000003  23-May-22  Unknown / Data Quality  14665
125616  E40000003  31-Jan-22  Unknown / Data Quality  14664
```

```
[838]: #See bottom 5 appointments
AD.sort_values(by="Appts", ascending=True).head()
```

```
[838]:      ICB_Code  ONS_Code      ICB_Name  \
59819      06H  E38000260  NHS Cambridgeshire and Peterborough ICB - 06H
98633      27D  E38000233      NHS Cheshire and Merseyside ICB - 27D
98635      27D  E38000233      NHS Cheshire and Merseyside ICB - 27D
116949     93C  E38000240      NHS North Central London ICB - 93C
52209      04Y  E38000028  NHS Staffordshire and Stoke-on-Trent ICB - 04Y

      ICB_ONS_Code  ONS_R_Code  Appt_Date      Duration  Appts
59819    E54000056  E40000007  03-Jun-22  21-30 Minutes      1
98633    E54000008  E40000010  27-Feb-22   6-10 Minutes      1
98635    E54000008  E40000010  27-Feb-22  21-30 Minutes      1
116949    E54000028  E40000003  10-Apr-22  21-30 Minutes      1
52209    E54000010  E40000011  01-May-22   6-10 Minutes      1
```

```
[839]: #View which ICB_ONS have most appts
ICB_Appts_ONS = (
    AD.groupby("ICB_ONS_Code")["Appts"]
      .sum()
      .reset_index()
)
```

```
ICB_Appts_ONS["Pct"] = ICB_Appts_ONS["Appts"] / ICB_Appts_ONS["Appts"].sum() * 100
```

```
ICB_Appts_ONS.sort_values("Pct", ascending=False).head(10)
```

```
[839]:
```

	ICB_ONS_Code	Appts	Pct
29	E54000050	9584943	5.7060
33	E54000054	8291440	4.9359
36	E54000057	7650341	4.5543
0	E54000008	7344348	4.3721
12	E54000027	6976986	4.1534
14	E54000029	5341883	3.1801
30	E54000051	5248579	3.1245
25	E54000042	5242840	3.1211
17	E54000032	5209641	3.1013
32	E54000053	5107481	3.0405

```
[840]: #Check the count of Duration x appt
AD["Duration"].value_counts().head(10)
```

```
[840]:
```

Duration	count
Unknown / Data Quality	20161
1-5 Minutes	19909
6-10 Minutes	19902
11-15 Minutes	19738
16-20 Minutes	19534
21-30 Minutes	19452
31-60 Minutes	19097

Name: count, dtype: int64

```
[841]: #Use groupby to view as new df and along with percentages
#Significant that Unknown / Data Quality is largest category
ICB_Appts = (
    AD.groupby("Duration")["Appts"]
      .sum()
      .reset_index()
)

ICB_Appts["Pct"] = ICB_Appts["Appts"] / ICB_Appts["Appts"].sum() * 100

ICB_Appts.sort_values("Pct", ascending=False).head(8)
```

```
[841]:
```

	Duration	Appts	Pct
6	Unknown / Data Quality	40284086	23.9814
5	6-10 Minutes	33800815	20.1218
0	1-5 Minutes	28600865	17.0263

1	11-15 Minutes	25160882	14.9784
2	16-20 Minutes	16004247	9.5274
3	21-30 Minutes	15026365	8.9453
4	31-60 Minutes	9103432	5.4193

```
[999]: #Notes from metadata...

#This field is recorded differently depending on the practice and system
↳supplier.
#Any appointments with a null duration or a duration of less than one minute or
↳greater
#than 60 minutes have been grouped into an ,Unknown / Data Quality Issue

#This makes it impossible to make a definitive decision about this data and
↳reduces quality significantly

# Create grouped duration categories with percentages
def group_duration(x):

    if x in ["1-5 Minutes", "6-10 Minutes", "11-15 Minutes"]:
        return "15 Minutes or Less"

    elif x == "Unknown / Data Quality":
        return "Unknown / Data Quality"

    else:
        return "16 Minutes to 1 Hour"

# Apply grouping
AD["Duration_Group"] = AD["Duration"].apply(group_duration)

# Aggregate appointments
ICB_Appts = (
    AD.groupby("Duration_Group")["Appts"]
    .sum()
    .reset_index()
)

# Calculate percentages
ICB_Appts["Pct"] = (
    ICB_Appts["Appts"] / ICB_Appts["Appts"].sum() * 100
)

# Sort largest first
ICB_Appts = ICB_Appts.sort_values("Pct", ascending=False)

# Create custom colors
```

```

colors = ["steelblue", "orange", "crimson"]

# Plot
plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data=ICB_Appts,
    x="Duration_Group",
    y="Pct",
    palette=colors
)

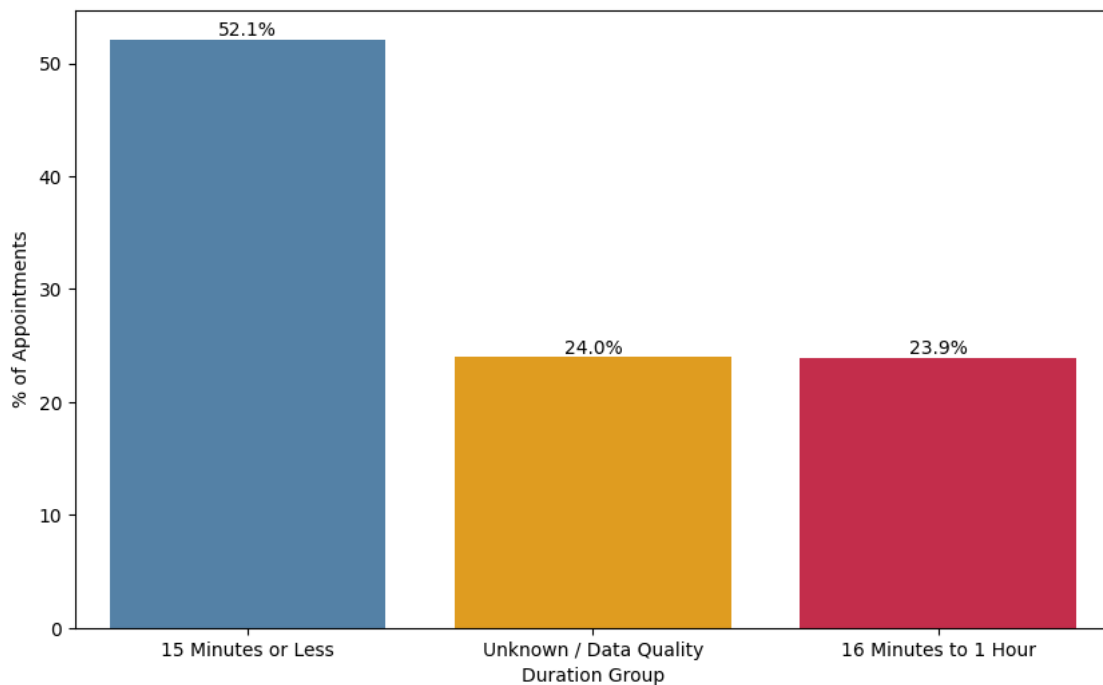
# Add percentage labels
for p in ax.patches:
    ax.annotate(
        f"{p.get_height():.1f}%",
        (p.get_x() + p.get_width()/2, p.get_height()),
        ha="center",
        va="bottom"
    )

plt.xlabel("Duration Group")
plt.ylabel("% of Appointments")

plt.xticks(rotation=0)

plt.show()

```




```
[843]: #Big jump in March and slightly smaller jump in May
#Massive decline in April
#Ensure datetime
AD["Appt_Date"] = pd.to_datetime(AD["Appt_Date"])

#Filter full range (Dec 2021 to Jun 2022)
df = AD[(AD["Appt_Date"] >= "2021-12-01") & (AD["Appt_Date"] <= "2022-06-30")].
    ↪copy()

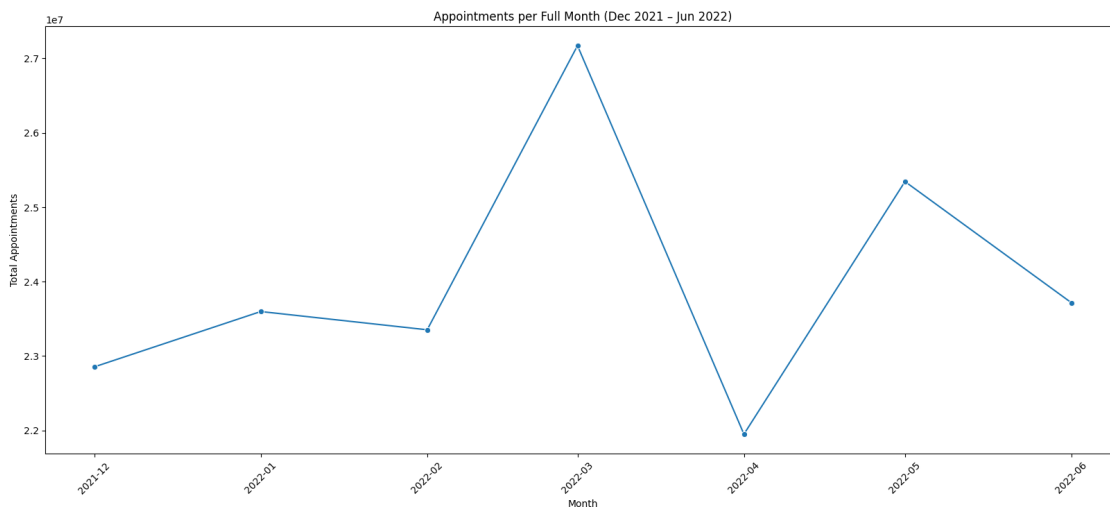
#Aggregate by full calendar month (Month Start ensures proper monthly bins)
monthly = df.set_index("Appt_Date")["Appts"].resample("MS").sum()

#Plot
plt.figure(figsize=(20, 8))
sns.lineplot(x=monthly.index, y=monthly.values, marker="o")

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Full Month (Dec 2021 - Jun 2022)")

plt.xticks(rotation=45)

plt.show()
```



```
[844]: #Grouped durations to make better sense of them - clearly skews towards shorter
    ↪appointments
AD["Appt_Date"] = pd.to_datetime(AD["Appt_Date"])
```

```

#Create grouped duration categories
def group_duration(x):

    if x in ["1-5 Minutes", "6-10 Minutes", "11-15 Minutes"]:
        return "15 Minutes or Less"

    elif x == "Unknown / Data Quality":
        return "Unknown / Data Quality"

    else:
        return "16 Minutes to 1 Hour"

#Apply grouping
AD["Duration_Group"] = AD["Duration"].apply(group_duration)

#Aggregate monthly appointments by duration group
monthly_duration = (
    AD.groupby([
        AD["Appt_Date"].dt.to_period("M").dt.to_timestamp(),
        "Duration_Group"
    ])["Appts"]
    .sum()
    .reset_index()
)

#Plot
plt.figure(figsize=(14, 7))

ax = sns.lineplot(
    data=monthly_duration,
    x="Appt_Date",
    y="Appts",
    hue="Duration_Group",
    marker="o"
)

#Format y-axis with actual values
ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.title("Monthly Appointments by Duration")
plt.xlabel("Month")
plt.ylabel("Number of Appointments")

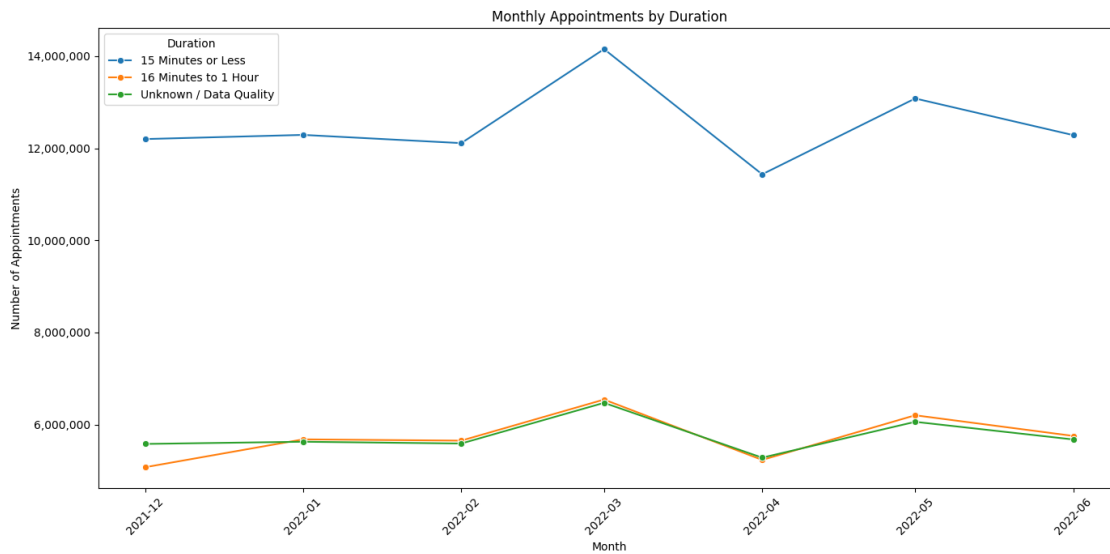
plt.xticks(rotation=45)

```

```
plt.legend(title="Duration")
```

```
plt.tight_layout()
```

```
plt.show()
```



```
[845]: # Select specific ICBs - higher unknown data quality than the average by 5
```

```
percent
```

```
selected_icbs = [
```

```
    "E54000057"
```

```
]
```

```
# Filter dataframe
```

```
df = AD[AD["ICB_ONS_Code"].isin(selected_icbs)].copy()
```

```
ICB_Appts = (
```

```
    df.groupby(["ICB_ONS_Code", "Duration"])["Appts"]
```

```
    .sum()
```

```
    .reset_index()
```

```
)
```

```
ICB_Appts["Pct"] = (
```

```
    ICB_Appts["Appts"] / ICB_Appts["Appts"].sum() * 100
```

```
)
```

```
# reorder columns
```

```
ICB_Appts = ICB_Appts.sort_values(
```

```
    ["ICB_ONS_Code", "Pct"],
```

```

        ascending=[True, False]
    )

ICB_Appts.sort_values("Pct", ascending=False).head(8)

```

```

[845]:
   ICB_ONS_Code      Duration  Appts    Pct
6    E54000057  Unknown / Data Quality  2155882  28.1802
5    E54000057      6-10 Minutes  1436981  18.7832
0    E54000057      1-5 Minutes  1137753  14.8719
1    E54000057     11-15 Minutes  1108594  14.4908
2    E54000057     16-20 Minutes   711168   9.2959
3    E54000057     21-30 Minutes   679574   8.8829
4    E54000057     31-60 Minutes   420389   5.4950

```

```

[846]: AD["Appt_Date"].value_counts().head(10)

```

```

[846]: Appt_Date
2021-12-01    742
2022-03-07    742
2022-04-06    742
2022-04-07    742
2022-04-08    742
2022-04-11    742
2022-04-12    742
2022-04-13    742
2022-04-14    742
2022-04-19    742
Name: count, dtype: int64

```

```

[847]: #106 Unique ICBs
AD["ICB_Name"].nunique()

```

```

[847]: 106

```

```

[848]: #This matches the ONS Code distinction
AD["ONS_Code"].nunique()

```

```

[848]: 106

```

```

[849]: #There are 42 ICB_ONS codes which is what matches through the 3 spreadsheets
AD["ICB_ONS_Code"].nunique()

```

```

[849]: 42

```

```

[850]: #There are 212 total dates included in this 7 month span
AD["Appt_Date"].nunique()

```

[850]: 212

```
[851]: #Over 7000 unique appointments recorded  
AD["Appts"].nunique()
```

[851]: 7611

```
[852]: #Appointments total: 167980692  
AD["Appts"].sum()
```

[852]: 167980692

```
[853]: #The largest category for duration is Unknown / Data Quality  
AD["Duration"].value_counts().head(10)
```

```
[853]: Duration  
Unknown / Data Quality    20161  
1-5 Minutes               19909  
6-10 Minutes              19902  
11-15 Minutes             19738  
16-20 Minutes             19534  
21-30 Minutes             19452  
31-60 Minutes             19097  
Name: count, dtype: int64
```

```
[854]: #Turning the date column into datetime quality  
AD["Appt_Date"] = pd.to_datetime(  
    AD["Appt_Date"],  
    format="%d-%b-%y"  
)
```

```
[855]: AD["Appt_Date"].value_counts().head(1)
```

```
[855]: Appt_Date  
2021-12-01    742  
Name: count, dtype: int64
```

```
[856]: AD["Appt_Date"].value_counts().min()
```

[856]: 128

```
[857]: AD["Appt_Date"].value_counts().max()
```

[857]: 742

```
[858]: #20k plus rows of Unknown + 40284086 total appts  
def contains_unknown(x):
```

```

"""Return True if Duration contains 'Unknown'."""

y = str(x).lower()

return "unknown" in y

# Apply the function
FD = AD["Duration"].apply(contains_unknown)

unknown_rows = AD[
    AD["Duration"].str.contains("Unknown", case=False, na=False)
]

print(unknown_rows)
print("\nTotal Appts:", unknown_rows["Appts"].sum())

```

	ICB_Code	ONS_Code	ICB_Name \
3	00L	E38000130	NHS North East and North Cumbria ICB - 00L
8	00L	E38000130	NHS North East and North Cumbria ICB - 00L
17	00L	E38000130	NHS North East and North Cumbria ICB - 00L
24	00L	E38000130	NHS North East and North Cumbria ICB - 00L
34	00L	E38000130	NHS North East and North Cumbria ICB - 00L
...	...	...	...
137758	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137765	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137772	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137779	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y
137786	X2C4Y	E38000254	NHS West Yorkshire ICB - X2C4Y

	ICB_ONS_Code	ONS_R_Code	Appt_Date	Duration	Appts \
3	E54000050	E40000012	2021-12-01	Unknown / Data Quality	1277
8	E54000050	E40000012	2021-12-02	Unknown / Data Quality	1391
17	E54000050	E40000012	2021-12-03	Unknown / Data Quality	1237
24	E54000050	E40000012	2021-12-04	Unknown / Data Quality	41
34	E54000050	E40000012	2021-12-06	Unknown / Data Quality	1502
...	...	...	...	...	...
137758	E54000054	E40000012	2022-06-25	Unknown / Data Quality	60
137765	E54000054	E40000012	2022-06-27	Unknown / Data Quality	2616
137772	E54000054	E40000012	2022-06-28	Unknown / Data Quality	2815
137779	E54000054	E40000012	2022-06-29	Unknown / Data Quality	2256
137786	E54000054	E40000012	2022-06-30	Unknown / Data Quality	2463

	Duration_Group
3	Unknown / Data Quality
8	Unknown / Data Quality
17	Unknown / Data Quality
24	Unknown / Data Quality

```

34      Unknown / Data Quality
...
137758 Unknown / Data Quality
137765 Unknown / Data Quality
137772 Unknown / Data Quality
137779 Unknown / Data Quality
137786 Unknown / Data Quality

```

[20161 rows x 9 columns]

Total Appts: 40284086

```

[859]: #Month and Year of total appointments
AD["year_month"] = AD["Appt_Date"].dt.to_period("M")

counts = AD.groupby("year_month")["Appts"].sum().sort_index()
counts.index = counts.index.astype(str)

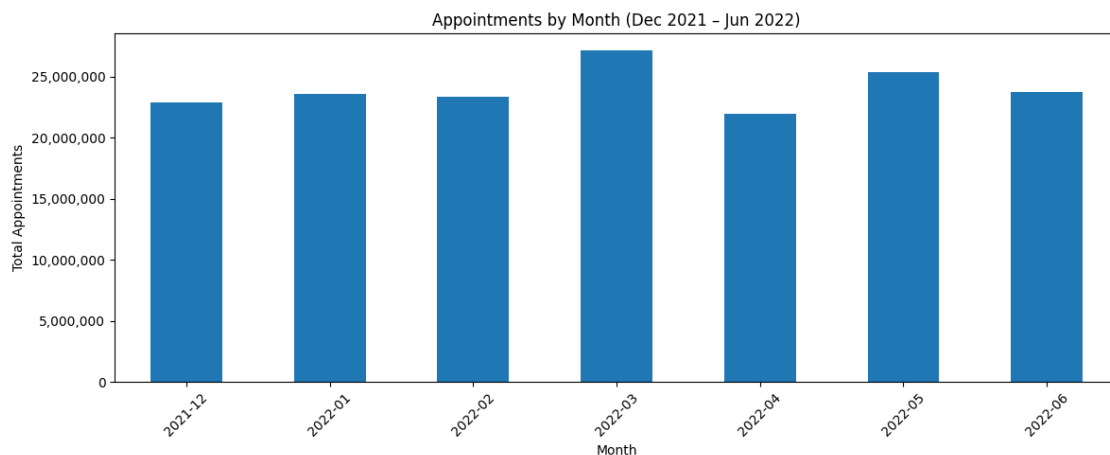
ax = counts.plot(kind="bar", figsize=(12, 5))

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments by Month (Dec 2021 - Jun 2022)")
plt.xticks(rotation=45)

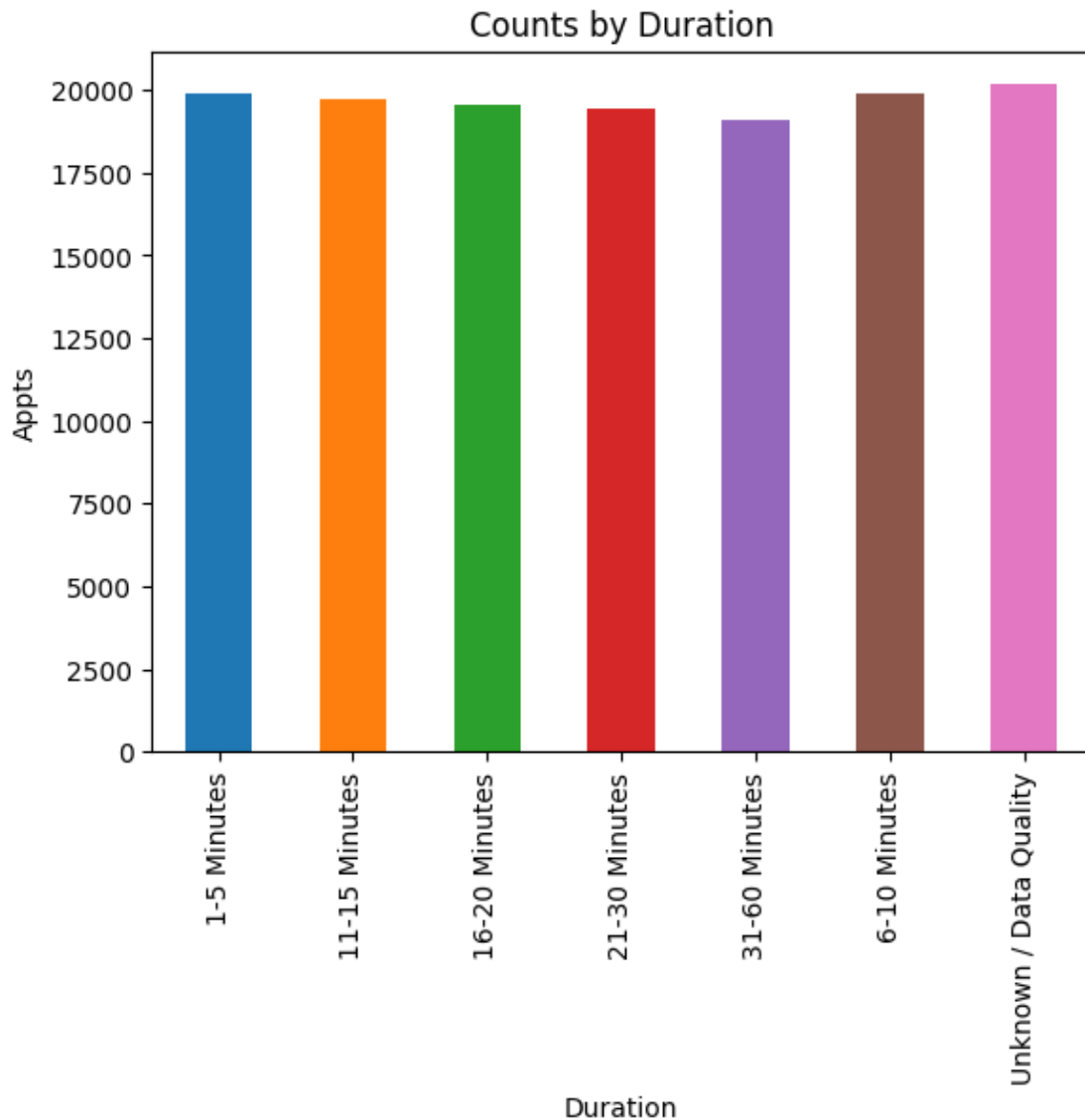
ax.yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.tight_layout()
plt.show()

```



```
[860]: #Can be considered random as to the duration of appt, no sig difference, major  
       ↳note is most common category is  
       #no data recorded which shows that it is not being made a priority to note -  
       ↳this may make the difference statistically  
       #As noted in the metadata this data quality is inherently low due to different  
       ↳ways of recording the duration of an appt  
       #...depending on practice/supplier system  
counts = AD["Duration"].value_counts().sort_index()  
  
colors = plt.cm.tab10(range(len(counts)))  
  
counts.plot(kind="bar", color=colors)  
  
plt.xlabel("Duration")  
plt.ylabel("Appts")  
plt.title("Counts by Duration")  
  
plt.show()
```

```
[861]: #From the month level the distribution looks quite stable for duration - would
        ↪ be useful to know
        #more about the "Unknown / Data Quality"
        AD["Month"] = AD["Appt_Date"].dt.month

        counts = AD.groupby(["Month", "Duration"]).size().unstack(fill_value=0)

        counts_pct = counts.div(counts.sum(axis=1), axis=0) * 100

        counts_pct.plot(
            kind="bar",
            stacked=True,
```

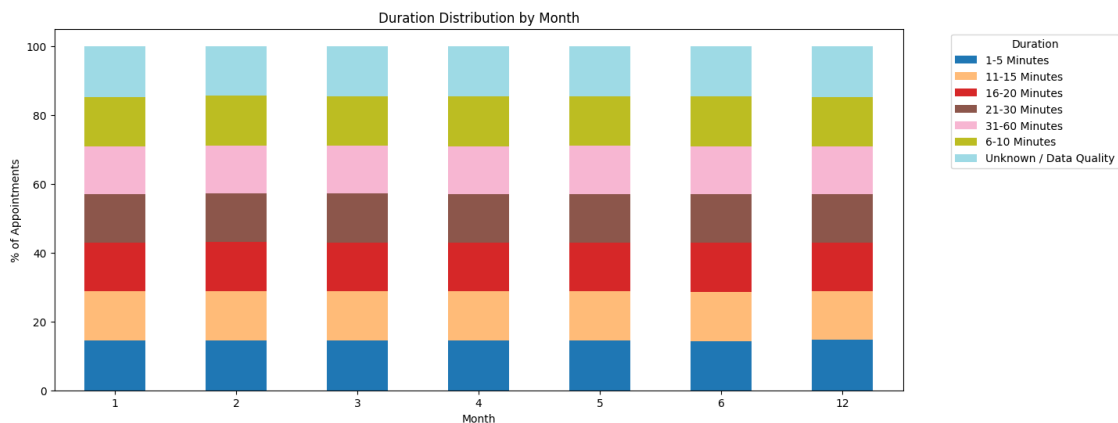
```

figsize=(14, 6),
colormap="tab20"
)

plt.xlabel("Month")
plt.ylabel("% of Appointments")
plt.title("Duration Distribution by Month")
plt.legend(title="Duration", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.xticks(rotation=0)

plt.show()

```



```

[862]: AD["Appt_Date"] = pd.to_datetime(AD["Appt_Date"])

#Make sure range is correct
df = AD[(AD["Appt_Date"] >= "2021-12-01") & (AD["Appt_Date"] <= "2022-06-30")].
    ↪copy()

#Create monthly totals per ICB
monthly = (
    df.groupby("ICB_ONS_Code")
      .resample("MS", on="Appt_Date")["Appts"]
      .sum()
      .reset_index()
)

#Plot
plt.figure(figsize=(20, 8))
sns.lineplot(
    data=monthly,
    x="Appt_Date",
    y="Appts",

```

```

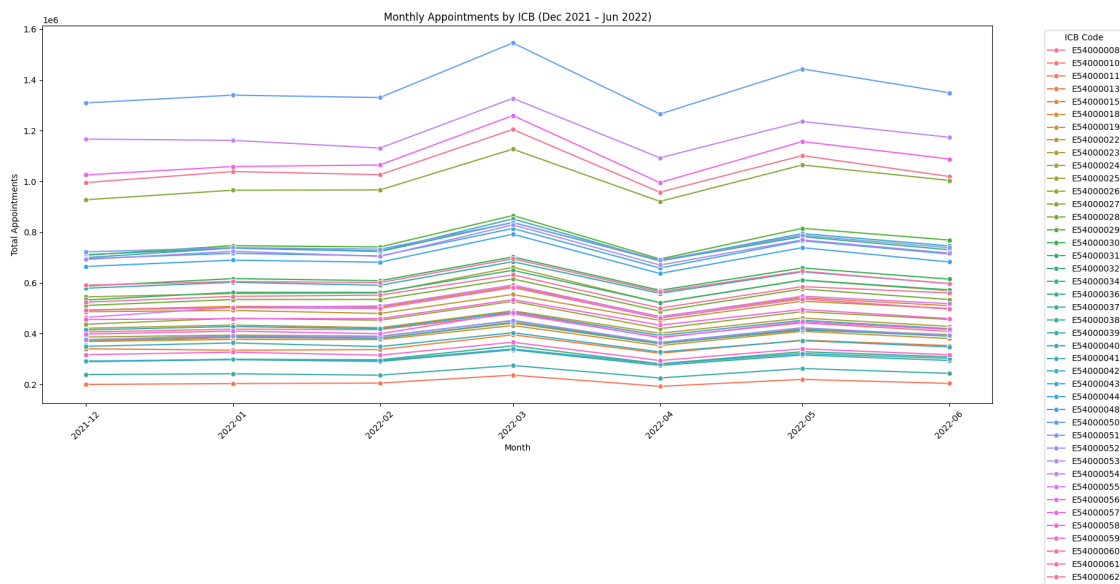
    hue="ICB_ONS_Code",
    marker="o"
)

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Monthly Appointments by ICB (Dec 2021 - Jun 2022)")
plt.xticks(rotation=45)

plt.legend(title="ICB Code", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.show()

```



```

[863]: #Top 5 ICBs show more drastic effect on March 2022 uptake in appts but same_
        ↪ patterns overall

# Ensure datetime
AD["Appt_Date"] = pd.to_datetime(AD["Appt_Date"])

# Filter date range
df = AD[(AD["Appt_Date"] >= "2021-12-01") & (AD["Appt_Date"] <= "2022-06-30")].
        ↪ copy()

# Keep top 5 ICBs in descending appointment order
top_icb = df.groupby("ICB_Name")["Appts"].sum().nlargest(5).index.tolist()

df = df[df["ICB_Name"].isin(top_icb)]

```

```

# Monthly aggregation
monthly = (
    df.groupby(["ICB_Name", pd.Grouper(key="Appt_Date", freq="MS")])["Appts"]
      .sum()
      .reset_index()
)

# Plot
plt.figure(figsize=(20, 8))

sns.lineplot(
    data=monthly,
    x="Appt_Date",
    y="Appts",
    hue="ICB_Name",
    hue_order=top_icb,    # <- fixes legend order
    marker="o"
)

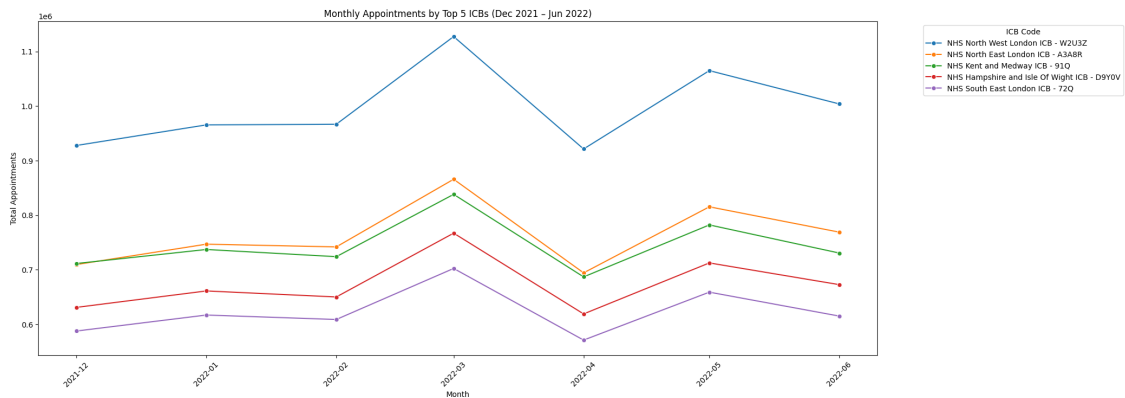
plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Monthly Appointments by Top 5 ICBs (Dec 2021 - Jun 2022)")

plt.xticks(rotation=45)

plt.legend(
    title="ICB Code",
    bbox_to_anchor=(1.05, 1),
    loc="upper left"
)

plt.show()

```



```
[864]: AD["Appts"].mean()
```

```
[864]: 1219.0800113213297
```

```
[865]: #Verification of percentages for duration - way too many are Unknown / Data  
↳Quality Issues  
duration_summary = (  
    AD.groupby("Duration")["Appts"]  
        .mean()  
        .reset_index(name="Mean_Appts")  
)  
  
# Add percentage column  
duration_summary["Pct"] = (  
    duration_summary["Mean_Appts"] /  
    duration_summary["Mean_Appts"].sum() * 100  
)  
  
# Sort largest first  
duration_summary = duration_summary.sort_values(  
    "Pct",  
    ascending=False  
)  
  
duration_summary
```

```
[865]:
```

	Duration	Mean_Appts	Pct
6	Unknown / Data Quality	1998.1194	23.5731
5	6-10 Minutes	1698.3627	20.0366
0	1-5 Minutes	1436.5797	16.9482
1	11-15 Minutes	1274.7432	15.0389
2	16-20 Minutes	819.3021	9.6658
3	21-30 Minutes	772.4843	9.1135
4	31-60 Minutes	476.6943	5.6239

```
[866]: #Order and percentage of duration stays the same throughout the months with  
↳unknown/ data quality top  
march_2022 = AD[  
    AD["Appt_Date"].dt.to_period("M") == "2022-03"  
]  
  
duration_summary = (  
    march_2022.groupby("Duration")["Appts"]  
        .mean()  
        .reset_index(name="Mean_Appts")  
)
```

```

# Add percentage column
duration_summary["Pct"] = (
    duration_summary["Mean_Appts"] /
    duration_summary["Mean_Appts"].sum() * 100
)

# Sort largest first
duration_summary = duration_summary.sort_values(
    "Pct",
    ascending=False
)

duration_summary

```

```

[866]:

```

	Duration	Mean_Appts	Pct
6	Unknown / Data Quality	2101.3221	23.5722
5	6-10 Minutes	1795.0242	20.1362
0	1-5 Minutes	1494.4781	16.7648
1	11-15 Minutes	1344.0400	15.0772
2	16-20 Minutes	863.1308	9.6824
3	21-30 Minutes	816.2199	9.1562
4	31-60 Minutes	500.1791	5.6109

3 National Categories Analysis

```

[867]: # View the DataFrame shape. Rows x columns.
NC.shape

```

```

[867]: (817394, 8)

```

```

[868]: #Mostly categorical data, only count of appts is int
print(NC.dtypes)

```

```

appointment_date      datetime64[ns]
icb_ons_code           object
sub_icb_location_name  object
service_setting        object
context_type           object
national_category      object
count_of_appointments  int64
appointment_month      object
dtype: object

```

```

[869]: #No Null values
NC.isnull().sum()

```

```
[869]: appointment_date      0
      icb_ons_code          0
      sub_icb_location_name  0
      service_setting        0
      context_type           0
      national_category       0
      count_of_appointments   0
      appointment_month       0
      dtype: int64
```

```
[870]: #No duplicates rows
      print("Duplicates:", NC.duplicated().sum())
```

Duplicates: 0

```
[871]: #View top of dataset
      NC.head()
```

```
[871]: appointment_date icb_ons_code      sub_icb_location_name \
0      2021-08-02      E54000050  NHS North East and North Cumbria ICB - 00L
1      2021-08-02      E54000050  NHS North East and North Cumbria ICB - 00L
2      2021-08-02      E54000050  NHS North East and North Cumbria ICB - 00L
3      2021-08-02      E54000050  NHS North East and North Cumbria ICB - 00L
4      2021-08-02      E54000050  NHS North East and North Cumbria ICB - 00L

      service_setting      context_type \
0  Primary Care Network  Care Related Encounter
1              Other  Care Related Encounter
2  General Practice  Care Related Encounter
3  General Practice  Care Related Encounter
4  General Practice  Care Related Encounter

      national_category  count_of_appointments \
0  Patient contact during Care Home Round      3
1              Planned Clinics      7
2              Home Visit      79
3  General Consultation Acute      725
4  Structured Medication Review      2

      appointment_month
0      2021-08
1      2021-08
2      2021-08
3      2021-08
4      2021-08
```

```
[872]: #View bottom of dataset
NC.tail()
```

```
[872]:      appointment_date icb_ons_code      sub_icb_location_name \
817389      2022-06-30      E54000054 NHS West Yorkshire ICB - X2C4Y
817390      2022-06-30      E54000054 NHS West Yorkshire ICB - X2C4Y
817391      2022-06-30      E54000054 NHS West Yorkshire ICB - X2C4Y
817392      2022-06-30      E54000054 NHS West Yorkshire ICB - X2C4Y
817393      2022-06-30      E54000054 NHS West Yorkshire ICB - X2C4Y

      service_setting      context_type \
817389 Extended Access Provision Care Related Encounter
817390 Extended Access Provision Care Related Encounter
817391 Extended Access Provision Care Related Encounter
817392 Extended Access Provision Care Related Encounter
817393 Extended Access Provision Care Related Encounter

      national_category count_of_appointments appointment_month
817389 Unplanned Clinical Activity              12      2022-06
817390      Planned Clinics                    4      2022-06
817391      Planned Clinical Procedure          92      2022-06
817392 General Consultation Routine              4      2022-06
817393 General Consultation Acute               19      2022-06
```

```
[873]: NC.describe()
```

```
[873]:      appointment_date count_of_appointments
count      817394      817394.0000
mean  2022-01-16 00:50:35.860796160      362.1837
min      2021-08-01 00:00:00      1.0000
25%      2021-10-25 00:00:00      7.0000
50%      2022-01-18 00:00:00      25.0000
75%      2022-04-07 00:00:00      128.0000
max      2022-06-30 00:00:00      16590.0000
std      NaN      1084.5766
```

```
[874]: #Rename columns to easier conventions
cols = NC.columns.tolist()
```

```
cols[0] = "Appt_Date"
cols[1] = "ICB_ONS"
cols[2] = "ICB_Name"
cols[3] = "Service"
cols[4] = "Context"
cols[5] = "Category"
cols[6] = "#_Appts"
cols[7] = "Month_Year"
```



```
NC.columns = cols
print(NC)
```

	Appt_Date	ICB_ONS	ICB_Name \
0	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
1	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
2	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
3	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
4	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
...	...	...	...
817389	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y
817390	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y
817391	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y
817392	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y
817393	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y

	Service	Context \
0	Primary Care Network	Care Related Encounter
1	Other	Care Related Encounter
2	General Practice	Care Related Encounter
3	General Practice	Care Related Encounter
4	General Practice	Care Related Encounter
...	...	...
817389	Extended Access Provision	Care Related Encounter
817390	Extended Access Provision	Care Related Encounter
817391	Extended Access Provision	Care Related Encounter
817392	Extended Access Provision	Care Related Encounter
817393	Extended Access Provision	Care Related Encounter

	Category	#_Appts	Month_Year
0	Patient contact during Care Home Round	3	2021-08
1	Planned Clinics	7	2021-08
2	Home Visit	79	2021-08
3	General Consultation Acute	725	2021-08
4	Structured Medication Review	2	2021-08
...	...	...	...
817389	Unplanned Clinical Activity	12	2022-06
817390	Planned Clinics	4	2022-06
817391	Planned Clinical Procedure	92	2022-06
817392	General Consultation Routine	4	2022-06
817393	General Consultation Acute	19	2022-06

[817394 rows x 8 columns]

```
[875]: #Check changes implemented
print(NC.columns)
```

```
Index(['Appt_Date', 'ICB_ONS', 'ICB_Name', 'Service', 'Context', 'Category',
      '#_Appts', 'Month_Year'],
      dtype='object')
```

```
[876]: #Sort by Category
NC.sort_values(by="Category", ascending=False).head()
```

```
[876]:      Appt_Date      ICB_ONS  \
235490  2022-05-30  E54000015
86319   2021-10-20  E54000048
298800  2022-06-13  E54000056
175258  2022-04-21  E54000051
807628  2021-08-11  E54000054

      ICB_Name      Service  \
235490  NHS Leicester Leicestershire and Rutland ICB -...  General Practice
86319   NHS Lancashire and South Cumbria ICB - 01K  General Practice
298800  NHS Cambridgeshire and Peterborough ICB - 06H  General Practice
175258  NHS Humber and North Yorkshire ICB - 02Y  General Practice
807628  NHS West Yorkshire ICB - X2C4Y  General Practice

      Context Category  #_Appts Month_Year
235490  Care Related Encounter  Walk-in      50    2022-05
86319   Care Related Encounter  Walk-in       6    2021-10
298800  Care Related Encounter  Walk-in    125    2022-06
175258  Care Related Encounter  Walk-in     16    2022-04
807628  Care Related Encounter  Walk-in     50    2021-08
```

```
[877]: #Sort by Category
NC.sort_values(by="Category", ascending=True).head()
```

```
[877]:      Appt_Date      ICB_ONS      ICB_Name  \
568842  2021-12-24  E54000031  NHS South West London ICB - 36L
535384  2021-09-06  E54000022  NHS Norfolk and Waveney ICB - 26A
423164  2022-02-10  E54000008  NHS Cheshire and Merseyside ICB - 12F
192425  2022-02-03  E54000051  NHS Humber and North Yorkshire ICB - 03K
699438  2021-12-22  E54000050  NHS North East and North Cumbria ICB - 99C

      Service      Context  \
568842  Primary Care Network  Care Related Encounter
535384   General Practice  Care Related Encounter
423164   General Practice  Care Related Encounter
192425  Primary Care Network  Care Related Encounter
699438   General Practice  Care Related Encounter

      Category  #_Appts Month_Year
568842  Care Home Needs Assessment & Personalised Care...      1    2021-12
```

535384	Care Home Needs Assessment & Personalised Care...	29	2021-09
423164	Care Home Needs Assessment & Personalised Care...	7	2022-02
192425	Care Home Needs Assessment & Personalised Care...	6	2022-02
699438	Care Home Needs Assessment & Personalised Care...	3	2021-12

```
[878]: #View top 10 ICBs
NC["ICB_Name"].value_counts().head(10)
```

```
[878]: ICB_Name
NHS North West London ICB - W2U3Z          13007
NHS Kent and Medway ICB - 91Q              12637
NHS Devon ICB - 15N                        12526
NHS Hampshire and Isle Of Wight ICB - D9Y0V 12171
NHS North East London ICB - A3A8R          11837
NHS Birmingham and Solihull ICB - 15E      11789
NHS Black Country ICB - D2P2L             11776
NHS Norfolk and Waveney ICB - 26A          11579
NHS Bath and North East Somerset Swindon and Wiltshire ICB - 92G 11545
NHS South West London ICB - 36L            10612
Name: count, dtype: int64
```

```
[879]: #View top 10 ICBs by appt value (easy to get confused between the previous and ↵
↵this)
NC.groupby("ICB_Name")["_Appts"].sum().sort_values(ascending=False).head(10)
```

```
[879]: ICB_Name
NHS North West London ICB - W2U3Z          12142390
NHS North East London ICB - A3A8R          9588891
NHS Kent and Medway ICB - 91Q              9286167
NHS Hampshire and Isle Of Wight ICB - D9Y0V 8288102
NHS South East London ICB - 72Q            7850170
NHS Devon ICB - 15N                        7447758
NHS South West London ICB - 36L            7155030
NHS Black Country ICB - D2P2L             7033637
NHS North Central London ICB - 93C        6747958
NHS Birmingham and Solihull ICB - 15E      6383746
Name: #_Appts, dtype: int64
```

```
[880]: #View top 10 busiest days
NC["Appt_Date"].value_counts().head(10)
```

```
[880]: Appt_Date
2022-06-28    3536
2022-06-21    3530
2022-05-17    3528
2022-05-10    3521
2022-06-07    3520
```

```

2022-05-24    3518
2022-06-14    3513
2022-06-22    3512
2022-05-25    3489
2022-06-29    3483
Name: count, dtype: int64

```

```

[881]: #Same as amount actual duration
NC["ICB_Name"].nunique()

```

```

[881]: 106

```

```

[882]: #Same as amount actual duration
NC["ICB_ONS"].nunique()

```

```

[882]: 42

```

```

[883]: #Significantly more unique dates than AD (makes sense given 4 more documented
↪months)
NC["Appt_Date"].nunique()

```

```

[883]: 334

```

```

[884]: #Total number of appts in NC
NC["#_Appts"].sum()

```

```

[884]: 296046770

```

```

[885]: #NC["Service"].nunique()
# 5 categories
NC["Service"].value_counts().head(10)

```

```

[885]: Service
General Practice      359274
Primary Care Network  183790
Other                 138789
Extended Access Provision  108122
Unmapped              27419
Name: count, dtype: int64

```

```

[886]: #NC["Context"].nunique()
# 3 values
NC["Context"].value_counts().head(10)

```

```

[886]: Context
Care Related Encounter    700481
Inconsistent Mapping      89494

```

```
Unmapped                27419
Name: count, dtype: int64
```

```
[887]: #NC["Category"].nunique() - numerous categories but top 5 are quite high
NC["Category"].value_counts().head(20)
```

```
[887]: Category
Inconsistent Mapping                89494
General Consultation Routine        89329
General Consultation Acute          84874
Planned Clinics                     76429
Clinical Triage                     74539
Planned Clinical Procedure          59631
Structured Medication Review        44467
Service provided by organisation external to the practice 43095
Home Visit                          41850
Unplanned Clinical Activity         40415
Patient contact during Care Home Round 28795
Unmapped                           27419
Care Home Visit                     26644
Social Prescribing Service          26492
Care Home Needs Assessment & Personalised Care and Support Planning 23505
Non-contractual chargeable work     20896
Walk-in                             14179
Group Consultation and Group Education 5341
Name: count, dtype: int64
```

```
[888]: #Equal number of unmapped rows across Service, Context, Category - confirmed
↳ same rows
NC[
    NC[["Service", "Context", "Category"]]
    .astype(str)
    .apply(lambda x: x.str.contains("Unmapped", na=False))
    .any(axis=1)
][["Service", "Context", "Category"]]
```

```
[888]:
```

	Service	Context	Category
6	Unmapped	Unmapped	Unmapped
36	Unmapped	Unmapped	Unmapped
65	Unmapped	Unmapped	Unmapped
92	Unmapped	Unmapped	Unmapped
130	Unmapped	Unmapped	Unmapped
...	...	...	...
817213	Unmapped	Unmapped	Unmapped
817253	Unmapped	Unmapped	Unmapped
817279	Unmapped	Unmapped	Unmapped
817305	Unmapped	Unmapped	Unmapped

817349 Unmapped Unmapped Unmapped

[27419 rows x 3 columns]

```
[889]: #20k plus of Unmapped
def contains_unknown(x):
    """Return True if Duration contains 'Unmapped'."""

    y = str(x).lower()

    return "Unmapped" in y

# Apply the function
FD = NC["Service"].apply(contains_unknown)

unknown_rows = NC[
    NC["Service"].str.contains("Unmapped", case=False, na=False)
]

print(unknown_rows)
print("\nTotal Appts:", unknown_rows["#_Appts"].sum())
```

	Appt_Date	ICB_ONS	ICB_Name \
6	2021-08-02	E54000050	NHS North East and North Cumbria ICB - 00L
36	2021-08-03	E54000050	NHS North East and North Cumbria ICB - 00L
65	2021-08-04	E54000050	NHS North East and North Cumbria ICB - 00L
92	2021-08-05	E54000050	NHS North East and North Cumbria ICB - 00L
130	2021-08-06	E54000050	NHS North East and North Cumbria ICB - 00L
...	...	...	...
817213	2022-06-25	E54000054	NHS West Yorkshire ICB - X2C4Y
817253	2022-06-27	E54000054	NHS West Yorkshire ICB - X2C4Y
817279	2022-06-28	E54000054	NHS West Yorkshire ICB - X2C4Y
817305	2022-06-29	E54000054	NHS West Yorkshire ICB - X2C4Y
817349	2022-06-30	E54000054	NHS West Yorkshire ICB - X2C4Y

	Service	Context	Category	#_Appts	Month_Year
6	Unmapped	Unmapped	Unmapped	372	2021-08
36	Unmapped	Unmapped	Unmapped	362	2021-08
65	Unmapped	Unmapped	Unmapped	336	2021-08
92	Unmapped	Unmapped	Unmapped	394	2021-08
130	Unmapped	Unmapped	Unmapped	302	2021-08
...	...	...	...	...	...
817213	Unmapped	Unmapped	Unmapped	1	2022-06
817253	Unmapped	Unmapped	Unmapped	225	2022-06
817279	Unmapped	Unmapped	Unmapped	176	2022-06
817305	Unmapped	Unmapped	Unmapped	146	2022-06
817349	Unmapped	Unmapped	Unmapped	186	2022-06

[27419 rows x 8 columns]

Total Appts: 11080810

```
[890]: #Change column to datetime format
NC["Appt_Date"] = pd.to_datetime(
    NC["Appt_Date"],
    format="%d-%b-%y"
)
```

```
[891]: #Checking display
NC["Appt_Date"].value_counts().head(1)
```

```
[891]: Appt_Date
2022-06-28    3536
Name: count, dtype: int64
```

```
[892]: NC["Appt_Date"].value_counts().min()
```

```
[892]: 164
```

```
[893]: NC["Appt_Date"].value_counts().max()
```

```
[893]: 3536
```

```
[894]: #Set seasons for later use if needed
conditions = [
    NC["Appt_Date"].dt.month.isin([6, 7, 8]),
    NC["Appt_Date"].dt.month.isin([9, 10, 11]),
    NC["Appt_Date"].dt.month.isin([12, 1, 2]),
    NC["Appt_Date"].dt.month.isin([3, 4, 5])
]

choices = ["Summer", "Autumn", "Winter", "Spring"]

NC["Season"] = np.select(conditions, choices)
```

```
[895]: #View in df
NC.head()
```

```
[895]:   Appt_Date  ICB_ONS  ICB_Name \
0 2021-08-02  E54000050  NHS North East and North Cumbria ICB - 00L
1 2021-08-02  E54000050  NHS North East and North Cumbria ICB - 00L
2 2021-08-02  E54000050  NHS North East and North Cumbria ICB - 00L
3 2021-08-02  E54000050  NHS North East and North Cumbria ICB - 00L
4 2021-08-02  E54000050  NHS North East and North Cumbria ICB - 00L
```

	Service	Context \
0	Primary Care Network	Care Related Encounter
1	Other	Care Related Encounter
2	General Practice	Care Related Encounter
3	General Practice	Care Related Encounter
4	General Practice	Care Related Encounter

	Category	#_Appts	Month_Year	Season
0	Patient contact during Care Home Round	3	2021-08	Summer
1	Planned Clinics	7	2021-08	Summer
2	Home Visit	79	2021-08	Summer
3	General Consultation Acute	725	2021-08	Summer
4	Structured Medication Review	2	2021-08	Summer

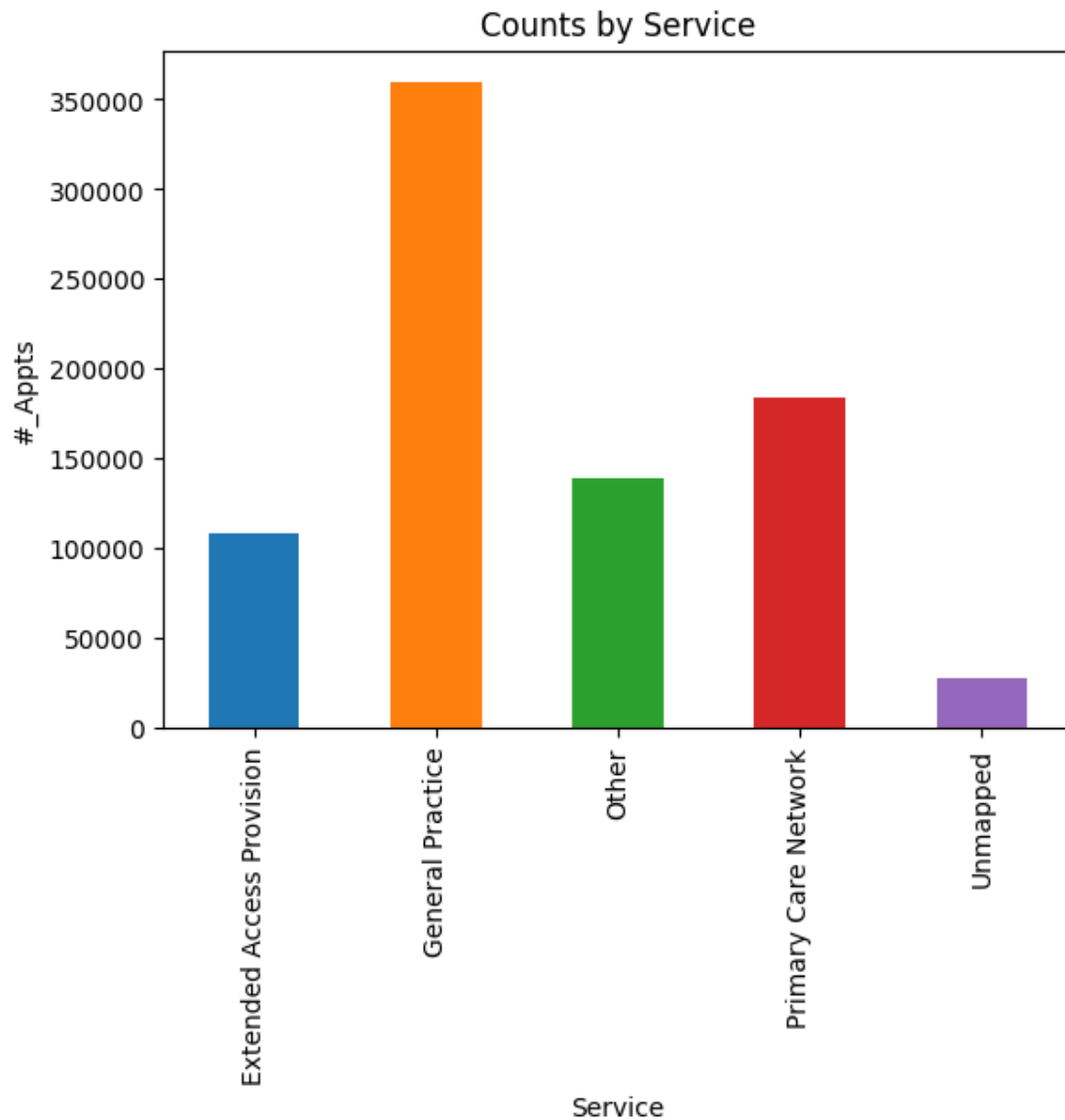
```
[896]: #GP is most common service by far followed by primary care network
counts = NC["Service"].value_counts().sort_index()

colors = plt.cm.tab10(range(len(counts)))

counts.plot(kind="bar", color=colors)

plt.xlabel("Service")
plt.ylabel("#_Appts")
plt.title("Counts by Service")

plt.show()
```

```
[897]: #Now that the different values are observed will rid of columns which are not GP  
NC = NC[  
    NC["Service"] == "General Practice"  
].copy()
```

```
[898]: #Checking if implemented  
NC["Service"].value_counts()
```

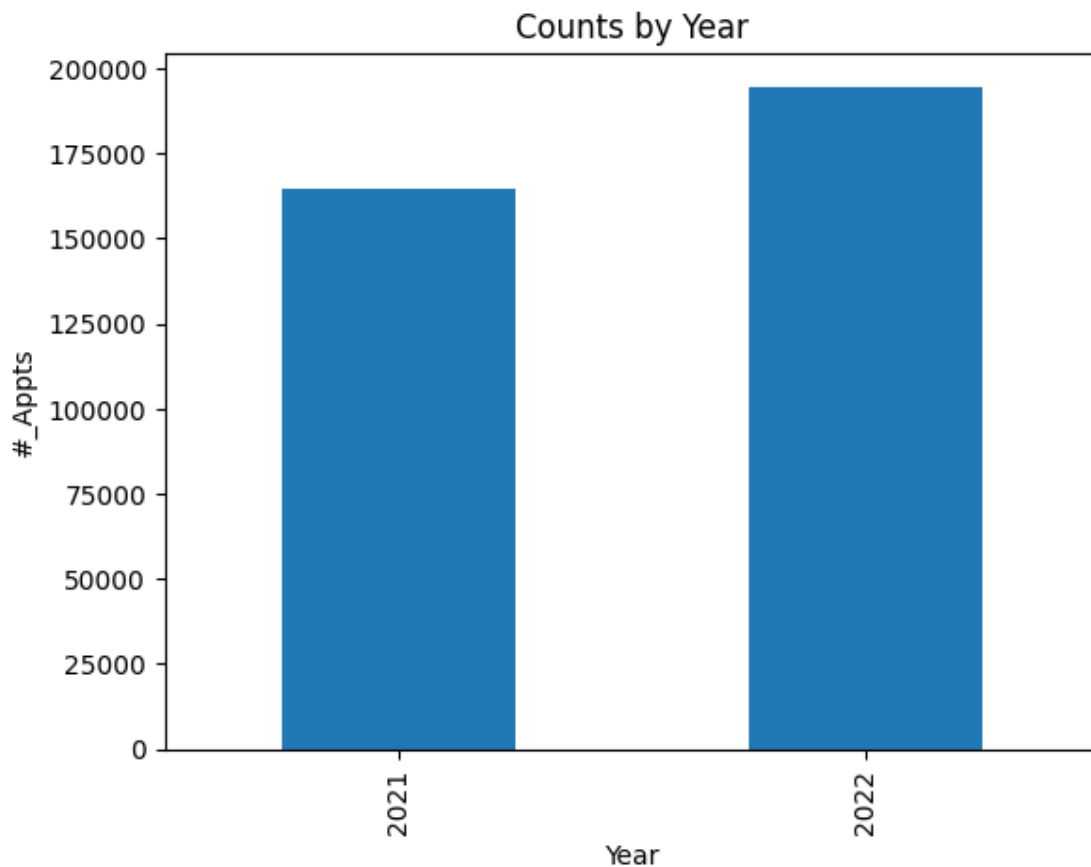
```
[898]: Service  
General Practice    359274  
Name: count, dtype: int64
```

```
[899]: #Slightly skewed for 2022
#5 months in 2021 / 6 months in 2022
NC["year"] = NC["Appt_Date"].dt.year

NC["year"].value_counts().sort_index().plot(kind="bar")

plt.xlabel("Year")
plt.ylabel("#_Appts")
plt.title("Counts by Year")

plt.show()
```



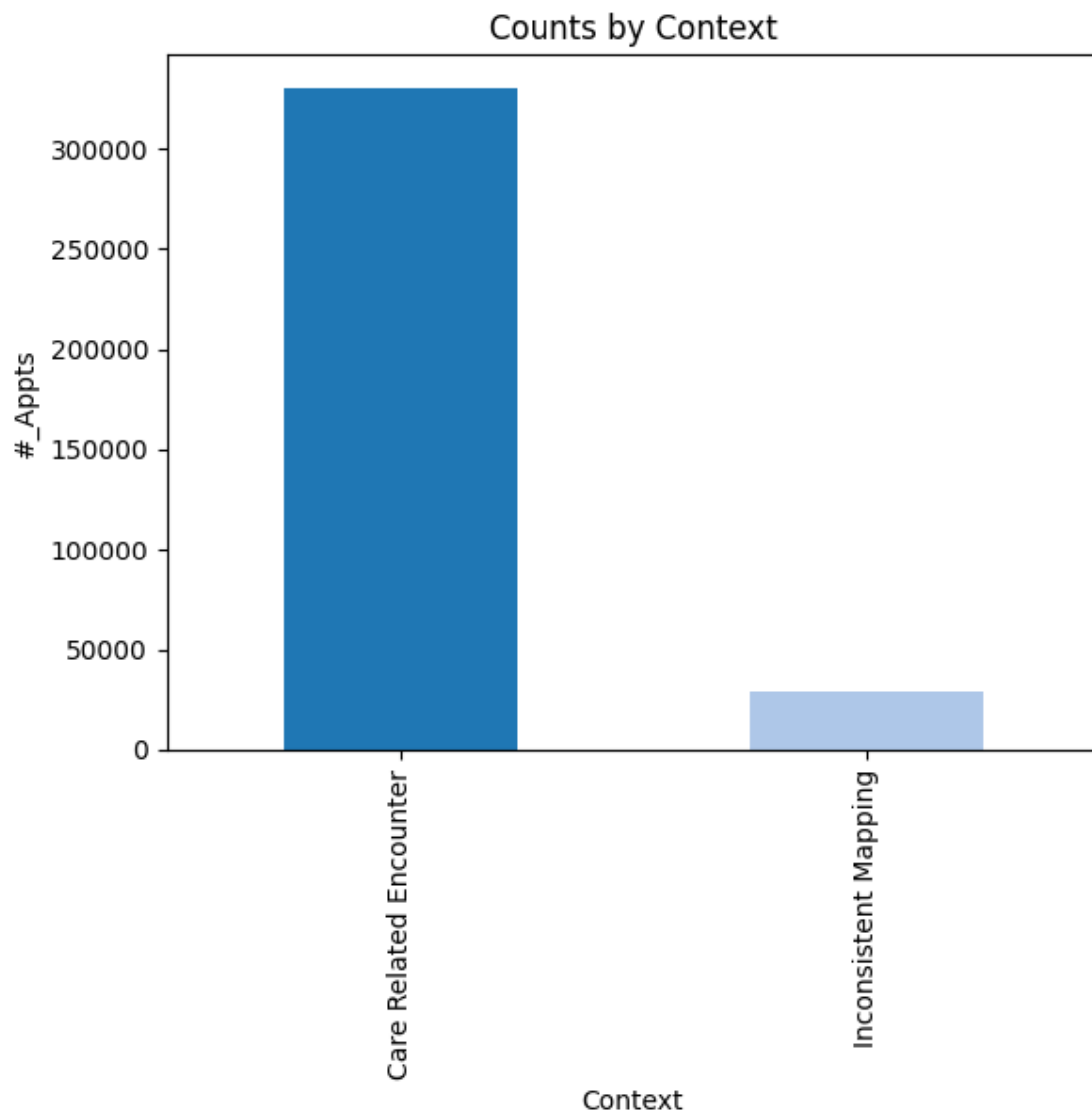
```
[900]: #Care related encounter is most appts - others don't provide much verifiable
info
counts = NC["Context"].value_counts().sort_index()

colors = plt.cm.tab20(range(12))

counts.plot(kind="bar", color=colors)
```

```
plt.xlabel("Context")
plt.ylabel("#_Appts")
plt.title("Counts by Context")

plt.show()
```



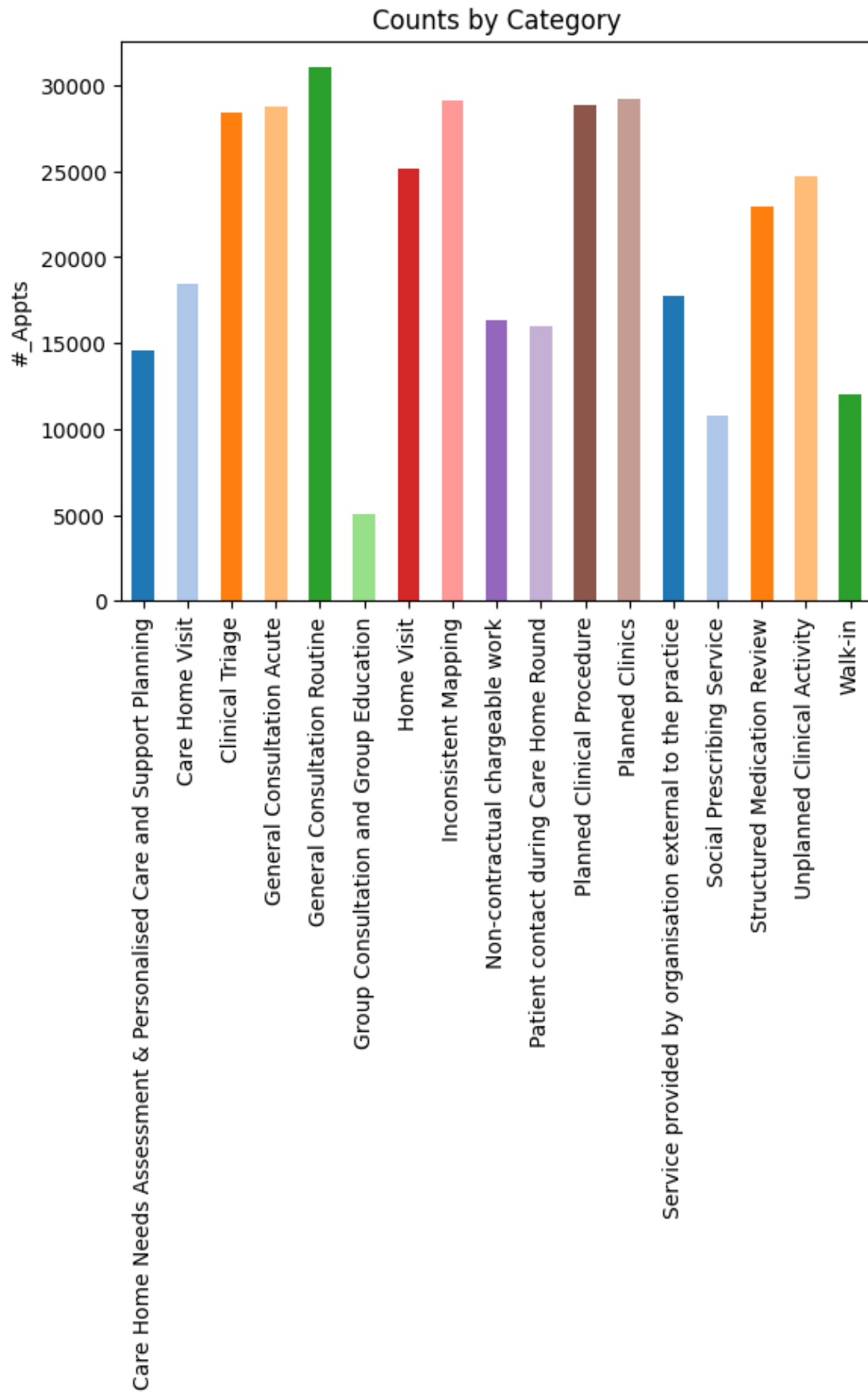
```
[901]: #Two categories of General consultation - would be useful to combine
counts = NC["Category"].value_counts().sort_index()

colors = plt.cm.tab20(range(12))
```

```
counts.plot(kind="bar", color=colors)

plt.xlabel("Category")
plt.ylabel("#_Apts")
plt.title("Counts by Category")

plt.show()
```



```
[902]: #Place into single category
NC["Category"] = NC["Category"].replace(
    {
        "General Consultation Acute": "General Consultation Acute/Routine",
        "General Consultation Routine": "General Consultation Acute/Routine"
    }
)

NC[NC["Category"] == "General Consultation Acute/Routine"]["Category"].
↪value_counts()
```

```
[902]: Category
General Consultation Acute/Routine    59771
Name: count, dtype: int64
```

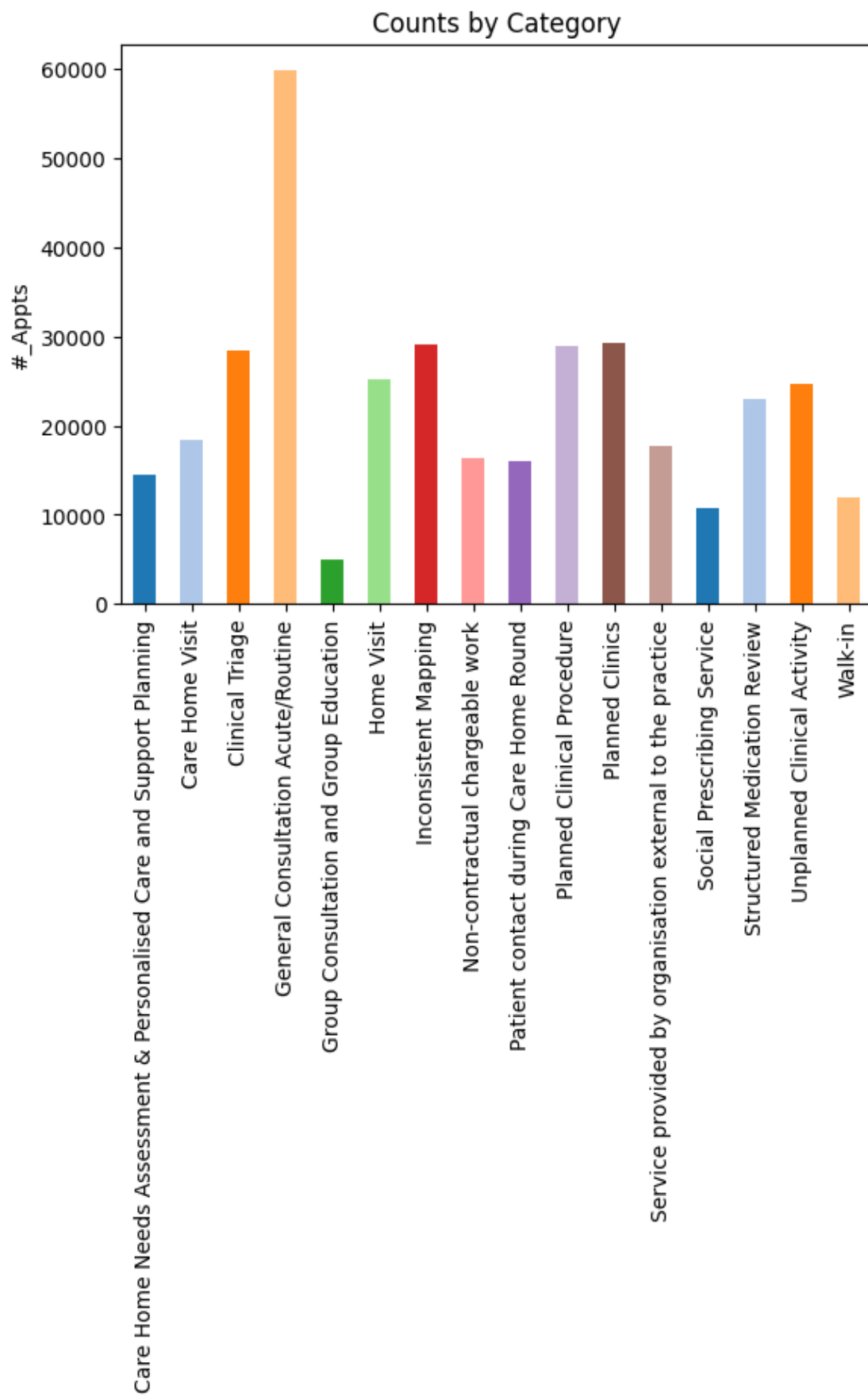
```
[903]: #Now more obvious that General consultations are the top category by far
counts = NC["Category"].value_counts().sort_index()

colors = plt.cm.tab20(range(12))

counts.plot(kind="bar", color=colors)

plt.xlabel("Category")
plt.ylabel("#_Appts")
plt.title("Counts by Category")

plt.show()
```



```
[904]: #Sep 2021 - Nov is highly impacted
#Big jump in March 2022 and slightly smaller jump in May 2022
#Massive decline in April

NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])

#Filter full range (Dec 2021 to Jun 2022)
df = NC[(NC["Appt_Date"] >= "2021-08-01") & (NC["Appt_Date"] <= "2022-06-30")]
    ↪copy()

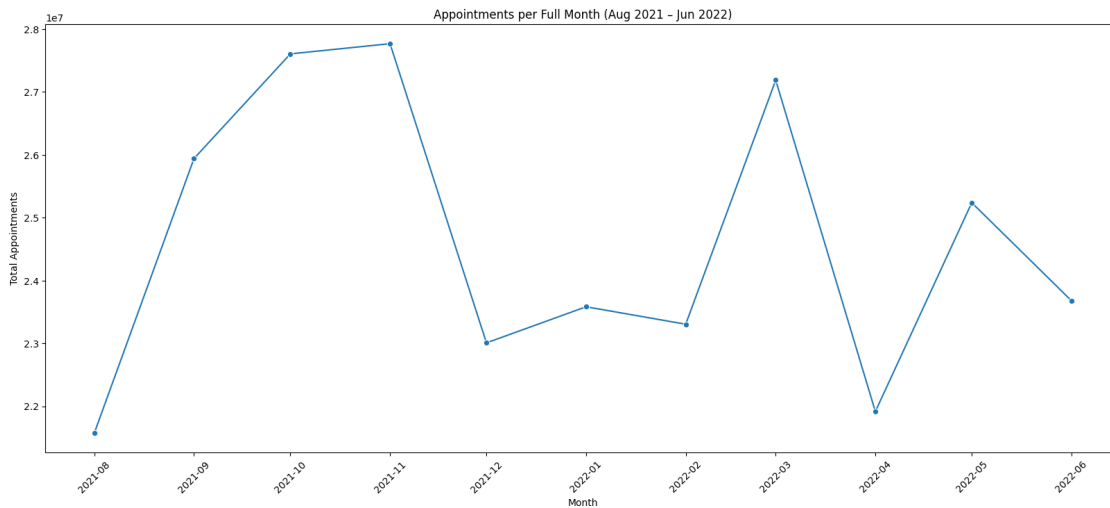
#Aggregate by full calendar month
monthly = df.set_index("Appt_Date")["#_Appts"].resample("MS").sum()

#Plot
plt.figure(figsize=(20, 8))
sns.lineplot(x=monthly.index, y=monthly.values, marker="o")

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Full Month (Aug 2021 - Jun 2022)")

plt.xticks(rotation=45)

plt.show()
```



```
[905]: # Ensure datetime
NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])
```



```

# Filter date range
df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

# SAFE monthly aggregation per ICB
monthly = (
    df.groupby([
        "ICB_ONS",
        pd.Grouper(key="Appt_Date", freq="MS")
    ])["#_Appts"]
    .sum()
    .reset_index()
)

# Plot
plt.figure(figsize=(20, 8))

sns.lineplot(
    data=monthly,
    x="Appt_Date",
    y="#_Appts",
    hue="ICB_ONS",
    marker="o"
)

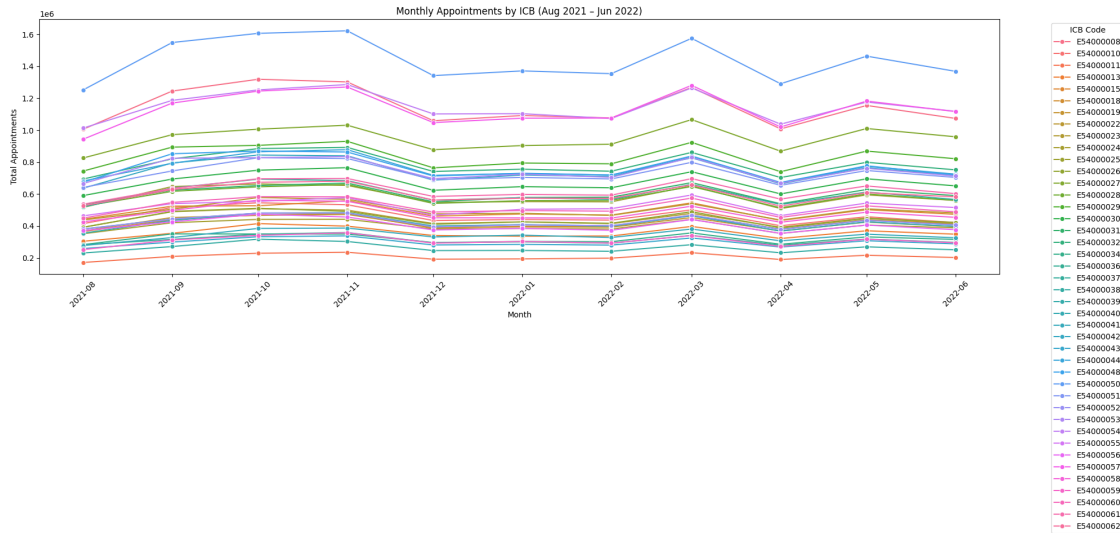
plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Monthly Appointments by ICB (Aug 2021 - Jun 2022)")

plt.xticks(rotation=45)

plt.legend(
    title="ICB Code",
    bbox_to_anchor=(1.05, 1),
    loc="upper left"
)

plt.tight_layout()
plt.show()

```



```
[1010]: NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])

# Filter date range
df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

# Top 5 ICBs
top_icb = (
    df.groupby("ICB_Name")["#_Appts"]
      .sum()
      .nlargest(3)
      .index
      .tolist()
)

df = df[df["ICB_Name"].isin(top_icb)]

# Monthly aggregation
monthly = (
    df.groupby(
        ["ICB_Name", pd.Grouper(key="Appt_Date", freq="MS")]
    )["#_Appts"]
      .sum()
      .reset_index()
)

# Identify TOP 3 lines (for labels only)
```

```

top3_icb = (
    df.groupby("ICB_Name")["#_Appts"]
      .sum()
      .nlargest(3)
      .index
      .tolist()
)

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly,
    x="Appt_Date",
    y="#_Appts",
    hue="ICB_Name",
    hue_order=top3_icb,
    marker="o"
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

for _, row in monthly.iterrows():

    if row["ICB_Name"] in top3_icb:
        ax.annotate(
            f"{row['#_Appts']:,.0f}",
            (row["Appt_Date"], row["#_Appts"]),
            textcoords="offset points",
            xytext=(0, 6),
            ha="center",
            fontsize=8
        )

plt.xlabel("Month")
plt.ylabel("Total Appointments")

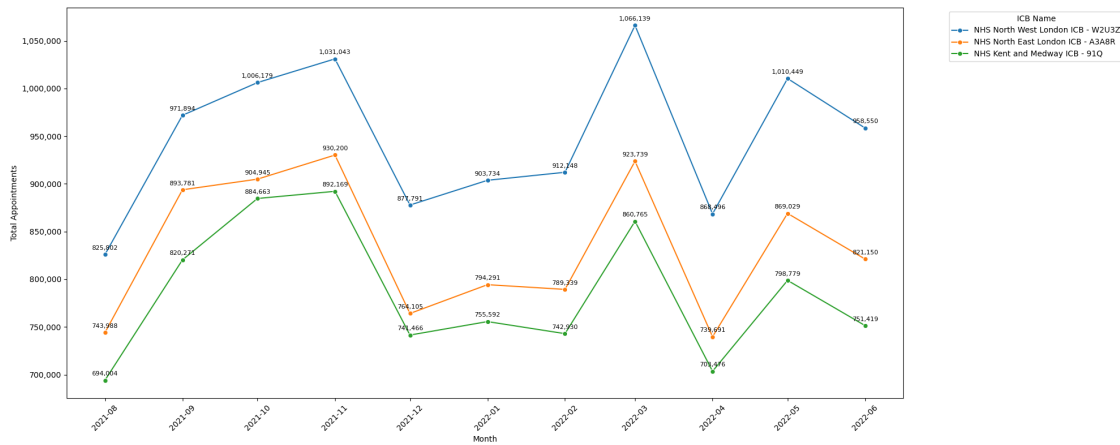
plt.xticks(rotation=45)

plt.legend(
    title="ICB Name",
    bbox_to_anchor=(1.05, 1),
    loc="upper left"
)

plt.tight_layout()

```

```
plt.show()
```



```
[1017]: #Now the bottom 3
NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])

df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

bottom_icb = (
    df.groupby("ICB_Name")["#_Appts"]
      .sum()
      .nsmallest(3)
      .index
      .tolist()
)

bottom_icb = sorted(
    bottom_icb,
    key=lambda x: df[df["ICB_Name"] == x]["#_Appts"].sum(),
    reverse=True
)

df = df[df["ICB_Name"].isin(bottom_icb)]

monthly = (
    df.groupby(
        ["ICB_Name", pd.Grouper(key="Appt_Date", freq="MS")]
    )["#_Appts"]
```

```

        .sum()
        .reset_index()
    )

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly,
    x="Appt_Date",
    y="#_Appts",
    hue="ICB_Name",
    hue_order=bottom_icb,
    marker="o"
)

ax.yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

for _, row in monthly.iterrows():
    ax.annotate(
        f"{row['_Appts']:, .0f}",
        (row["Appt_Date"], row["_Appts"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=8
    )

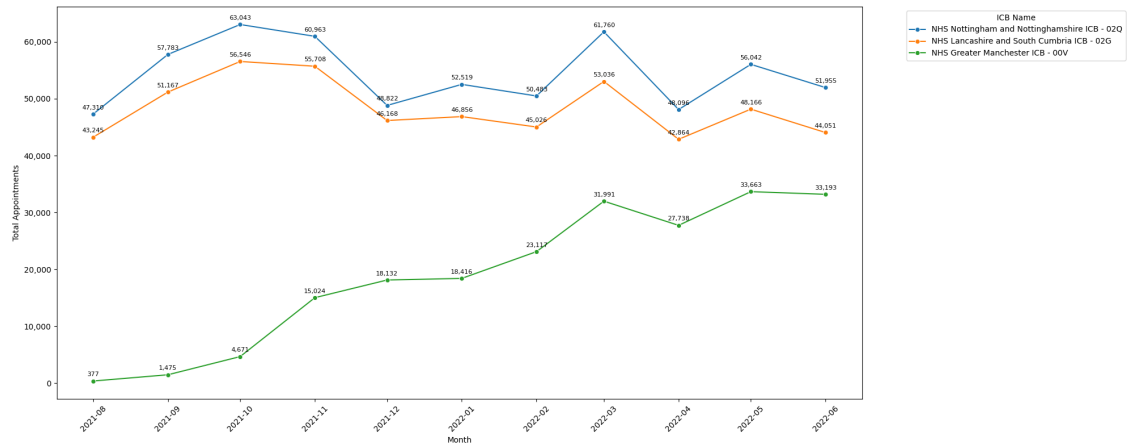
plt.xlabel("Month")
plt.ylabel("Total Appointments")

plt.xticks(rotation=45)

plt.legend(
    title="ICB Name",
    bbox_to_anchor=(1.05, 1),
    loc="upper left"
)

plt.tight_layout()
plt.show()

```



```
[907]: NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])

#Get weekday
NC["Weekday"] = NC["Appt_Date"].dt.day_name()

#Total appointments by weekday
weekday_appts = (
    NC.groupby("Weekday")["#_Appts"]
      .sum()
      .reindex([
          "Monday", "Tuesday", "Wednesday",
          "Thursday", "Friday", "Saturday", "Sunday"
      ])
      .reset_index()
)

weekday_appts["Pct"] = (
    weekday_appts["#_Appts"] / weekday_appts["#_Appts"].sum() * 100
)

saturday_ratio = weekday_appts.loc[
    weekday_appts["Weekday"] == "Saturday", "Pct"
].values[0]

#Also noted number of saturday appointments for later use
print(f"Saturday share of total appointments: {saturday_ratio:.2f}%")

# Plot
plt.figure(figsize=(10, 5))

sns.barplot(
    data=weekday_appts,
```

```

    x="Weekday",
    y="Pct"
)

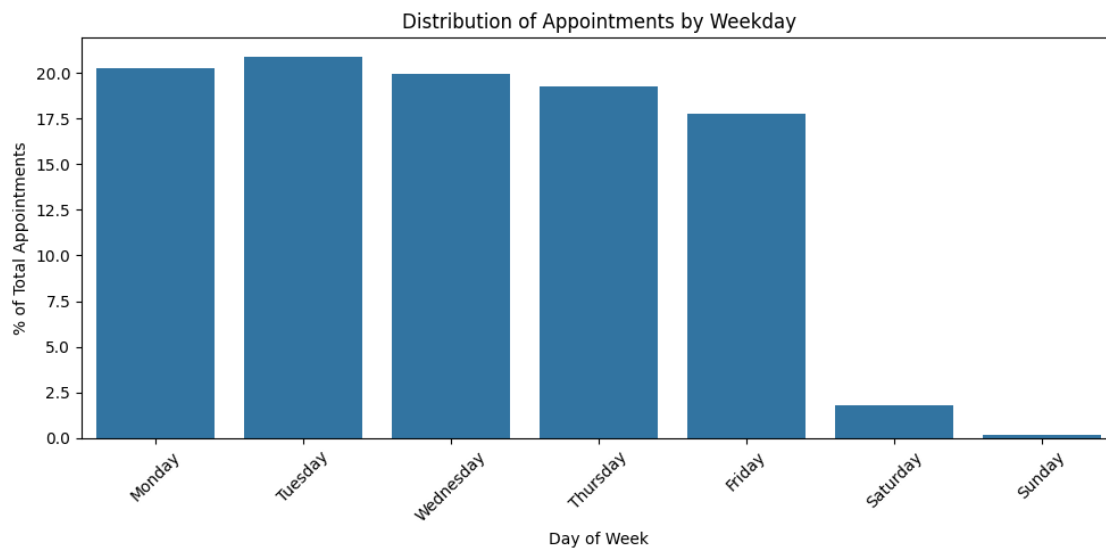
plt.title("Distribution of Appointments by Weekday")
plt.ylabel("% of Total Appointments")
plt.xlabel("Day of Week")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

weekday_appts

```

Saturday share of total appointments: 1.79%



```

[907]:
  Weekday  #_Appts  Pct
0  Monday  54794619 20.2335
1  Tuesday  56529751 20.8742
2  Wednesday 53980690 19.9329
3  Thursday 52084657 19.2328
4  Friday  48099601 17.7613
5  Saturday 4856121  1.7932
6  Sunday   466252  0.1722

```

```

[908]: #Unmapped (11,080,810): No recorded category against an appointment slot type.
        ↳ This could be due to an error
        #receiving the data, or an appointment type has not been mapped.

```

```

#Inconsistent Mapping (28,720,894): Care Related Encounter but does not fit
↳into any other category /
#Appointment types not mapped to a Care Related Encounter

service_context_pivot = (
    NC.pivot_table(
        index="Service",
        columns="Context",
        values="#_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

service_context_pivot

```

[908]: Context Care Related Encounter Inconsistent Mapping
Service
General Practice 245814216 24997475

```

[909]: service_context_pivot = (
    NC.pivot_table(
        index="Service",
        columns="Context",
        values="#_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

service_context_pivot_pct = (
    service_context_pivot.div(service_context_pivot.sum(axis=1), axis=0) * 100
).round(2)

service_context_pivot_pct = service_context_pivot_pct.apply(
    lambda row: row.sort_values(ascending=False),
    axis=1
)

service_context_pivot_pct
#Care Related Encounter but does not fit into any other category
#Note that for "Other" - Appointments delivered in the practice by another
↳provider.
#"Extended Access Provision" - Appointments commissioned as part of extended
↳access contractual

```


[909]:	Context	Care Related Encounter	Inconsistent Mapping
	Service		
	General Practice	90.7700	9.2300

```
[910]: #Table showing percentage of Service x category
service_category_pivot = (
    NC.pivot_table(
        index="Service",
        columns="Category",
        values="#_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

service_category_pivot_pct = (
    service_category_pivot.div(service_category_pivot.sum(axis=1), axis=0) * 100
).round(2)

service_category_pivot_pct = service_category_pivot_pct.apply(
    lambda row: row.sort_values(ascending=False),
    axis=1
)

service_category_pivot_pct
```

[910]:	Category	General Consultation	Acute/Routine	Clinical Triage	\
	Service				
	General Practice		53.7000	14.8600	
	Category	Planned Clinics	Inconsistent Mapping	\	
	Service				
	General Practice	9.7400	9.2300		
	Category	Planned Clinical Procedure	Unplanned Clinical Activity	\	
	Service				
	General Practice	9.2300	1.0600		
	Category	Home Visit	Structured Medication Review	Care Home Visit	\
	Service				
	General Practice	0.7500	0.4800	0.2000	
	Category	Patient contact during Care Home Round	\		
	Service				
	General Practice	0.1900			
	Category	Service provided by organisation external to the practice	\		

Service			
General Practice			0.1700
Category	Walk-in \		
Service			
General Practice		0.1300	
Category	Care Home Needs Assessment & Personalised Care and Support		
Planning \			
Service			
General Practice			0.1200
Category	Social Prescribing Service	Non-contractual chargeable work \	
Service			
General Practice		0.0700	0.0400
Category	Group Consultation and Group Education		
Service			
General Practice			0.0200

```
[911]: #Top ICB x Service - Counts + Percentages
icb_service_counts = (
    NC.pivot_table(
        index="ICB_Name",
        columns="Service",
        values="#_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

icb_service_counts["Total_Appts"] = icb_service_counts.sum(axis=1)

service_cols = icb_service_counts.columns.drop("Total_Appts")

icb_service_pct = (
    icb_service_counts[service_cols]
    .div(icb_service_counts["Total_Appts"], axis=0) * 100
)

icb_final = pd.concat(
    [icb_service_counts, icb_service_pct.add_suffix("_Pct")],
    axis=1
)

icb_final_top5 = icb_final.sort_values("Total_Appts", ascending=False).head(5)
```

```
icb_final_top5
```

```
[911]: Service                                General Practice  Total_Appts  \
ICB_Name
NHS North West London ICB - W2U3Z          10432225      10432225
NHS North East London ICB - A3A8R           9174258       9174258
NHS Kent and Medway ICB - 91Q                8645534       8645534
NHS Hampshire and Isle Of Wight ICB - D9Y0V  7407509       7407509
NHS South East London ICB - 72Q              7395389       7395389

Service                                General Practice_Pct
ICB_Name
NHS North West London ICB - W2U3Z          100.0000
NHS North East London ICB - A3A8R           100.0000
NHS Kent and Medway ICB - 91Q                100.0000
NHS Hampshire and Isle Of Wight ICB - D9Y0V  100.0000
NHS South East London ICB - 72Q              100.0000
```

```
[912]: NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])
```

```
df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

monthly_service = (
    df.set_index("Appt_Date")
      .groupby("Service")["#_Appts"]
      .resample("MS")
      .sum()
      .reset_index()
)

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly_service,
    x="Appt_Date",
    y="#_Appts",
    hue="Service",
    marker="o"
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)
```

```

#Label General Practice
gp_data = monthly_service[monthly_service["Service"] == "General Practice"]

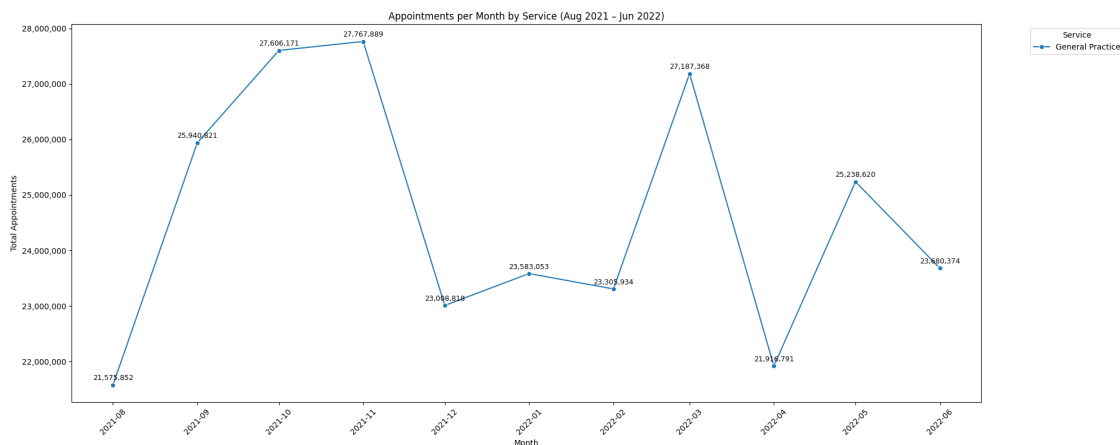
for _, row in gp_data.iterrows():
    ax.annotate(
        f"{row['#_Appts']:, .0f}",
        (row["Appt_Date"], row["#_Appts"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=9
    )

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Service (Aug 2021 - Jun 2022)")
plt.xticks(rotation=45)

plt.legend(title="Service", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



[913]: *#Note from figure - GPs are far overworked compared to other types of services offered*

[914]: *#Care related encounters are significantly the highest*

```

context_summary = (
    NC.groupby("Context")["#_Appts"]
    .sum()
    .sort_values(ascending=False)
)

```

```

        .head(10)
        .reset_index()
    )

    context_summary["Pct"] = (
        context_summary["_Appts"] /
        context_summary["_Appts"].sum() * 100
    )

    context_summary

```

```

[914]:
           Context    #_Appts    Pct
0  Care Related Encounter  245814216  90.7694
1   Inconsistent Mapping   24997475   9.2306

```

```

[915]: #Top 5 ICB x Context
icb_context_counts = (
    NC.pivot_table(
        index="ICB_Name",
        columns="Context",
        values="_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

icb_context_counts["Total_Appts"] = icb_context_counts.sum(axis=1)

context_cols = icb_context_counts.columns.drop("Total_Appts")

icb_context_pct = (
    icb_context_counts[context_cols]
    .div(icb_context_counts["Total_Appts"], axis=0) * 100
)

icb_final = pd.concat(
    [icb_context_counts, icb_context_pct.add_suffix("_Pct")],
    axis=1
)

icb_final_top5 = icb_final.sort_values("Total_Appts", ascending=False).head(5)

icb_final_top5

```

```

[915]: Context                                Care Related Encounter \
ICB_Name
NHS North West London ICB - W2U3Z                                9375480

```

NHS North East London ICB - A3A8R	8006279
NHS Kent and Medway ICB - 91Q	7589145
NHS Hampshire and Isle Of Wight ICB - D9Y0V	6663055
NHS South East London ICB - 72Q	6552250

Context	Inconsistent Mapping \
ICB_Name	
NHS North West London ICB - W2U3Z	1056745
NHS North East London ICB - A3A8R	1167979
NHS Kent and Medway ICB - 91Q	1056389
NHS Hampshire and Isle Of Wight ICB - D9Y0V	744454
NHS South East London ICB - 72Q	843139

Context	Total_Appts \
ICB_Name	
NHS North West London ICB - W2U3Z	10432225
NHS North East London ICB - A3A8R	9174258
NHS Kent and Medway ICB - 91Q	8645534
NHS Hampshire and Isle Of Wight ICB - D9Y0V	7407509
NHS South East London ICB - 72Q	7395389

Context	Care Related Encounter_Pct \
ICB_Name	
NHS North West London ICB - W2U3Z	89.8704
NHS North East London ICB - A3A8R	87.2690
NHS Kent and Medway ICB - 91Q	87.7811
NHS Hampshire and Isle Of Wight ICB - D9Y0V	89.9500
NHS South East London ICB - 72Q	88.5991

Context	Inconsistent Mapping_Pct
ICB_Name	
NHS North West London ICB - W2U3Z	10.1296
NHS North East London ICB - A3A8R	12.7310
NHS Kent and Medway ICB - 91Q	12.2189
NHS Hampshire and Isle Of Wight ICB - D9Y0V	10.0500
NHS South East London ICB - 72Q	11.4009

```
[916]: NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])
```

```
df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

monthly_service = (
    df.set_index("Appt_Date")
    .groupby("Context")["#_Appts"]
```

```

        .resample("MS")
        .sum()
        .reset_index()
    )

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly_service,
    x="Appt_Date",
    y="#_Appts",
    hue="Context",
    marker="o"
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

#Annotate Care Related Encounters
care_data = monthly_service[
    monthly_service["Context"] == "Care Related Encounter"
]

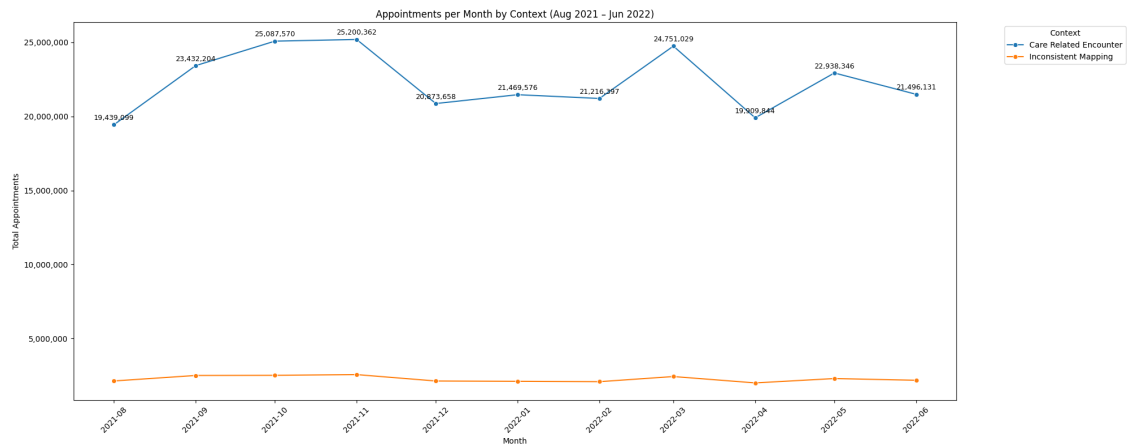
for _, row in care_data.iterrows():
    ax.annotate(
        f"{row['_Appts']:, .0f}",
        (row["Appt_Date"], row["_Appts"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=9
    )

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Context (Aug 2021 - Jun 2022)")
plt.xticks(rotation=45)

plt.legend(title="Context", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



[917]: *#Aligns very closely with GP appointments*

[918]: *#Count/Percentage of Categories*

```
category_summary = (
    NC.groupby("Category")["#_Appts"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

category_summary["Pct"] = (
    category_summary["#_Appts"] /
    category_summary["#_Appts"].sum() * 100
)

category_summary
```

```
[918]:
```

	Category	#_Appts	Pct
0	General Consultation Acute/Routine	145418402	53.9977
1	Clinical Triage	40246576	14.9446
2	Planned Clinics	26376358	9.7942
3	Inconsistent Mapping	24997475	9.2822
4	Planned Clinical Procedure	24986338	9.2781
5	Unplanned Clinical Activity	2872873	1.0668
6	Home Visit	2040497	0.7577
7	Structured Medication Review	1312506	0.4874
8	Care Home Visit	536382	0.1992
9	Patient contact during Care Home Round	517649	0.1922


```
[919]: #Count Service x Category
service_category_pivot = (
    NC.pivot_table(
        index="Service",
        columns="Category",
        values="#_Appts",
        aggfunc="sum",
        fill_value=0
    )
)

service_category_pivot
```

```
[919]: Category          Care Home Needs Assessment & Personalised Care and Support
Planning \
Service
General Practice          316586

Category          Care Home Visit  Clinical Triage \
Service
General Practice          536382          40246576

Category          General Consultation Acute/Routine \
Service
General Practice          145418402

Category          Group Consultation and Group Education  Home Visit \
Service
General Practice          59085          2040497

Category          Inconsistent Mapping  Non-contractual chargeable work \
Service
General Practice          24997475          116053

Category          Patient contact during Care Home Round \
Service
General Practice          517649

Category          Planned Clinical Procedure  Planned Clinics \
Service
General Practice          24986338          26376358

Category          Service provided by organisation external to the practice \
Service
General Practice          469974

Category          Social Prescribing Service  Structured Medication Review \
```

Service		
General Practice	181371	1312506
Category	Unplanned Clinical Activity	Walk-in
Service		
General Practice	2872873	363566

```
[920]: NC["Appt_Date"] = pd.to_datetime(NC["Appt_Date"])

df = NC[
    (NC["Appt_Date"] >= "2021-08-01") &
    (NC["Appt_Date"] <= "2022-06-30")
].copy()

monthly_service = (
    df.set_index("Appt_Date")
      .groupby("Category")["#_Appts"]
      .resample("MS")
      .sum()
      .reset_index()
)

category_order = (
    monthly_service.groupby("Category")["#_Appts"]
      .sum()
      .sort_values(ascending=False)
      .index
      .tolist()
)

#Add distinct colours
palette = sns.color_palette("tab10", n_colors=len(category_order))

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly_service,
    x="Appt_Date",
    y="#_Appts",
    hue="Category",
    hue_order=category_order,
    palette=palette,
    marker="o"
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
```

```

)

#Label top 3 categories
top3_data = monthly_service[monthly_service["Category"].isin(top3)]

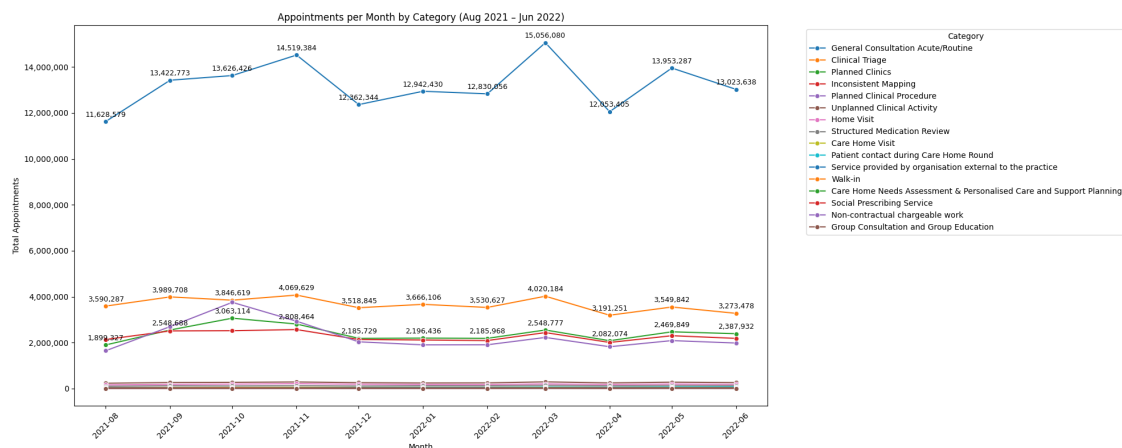
for _, row in top3_data.iterrows():
    ax.annotate(
        f"{row['#_Appts']:.0f}",
        (row["Appt_Date"], row["#_Appts"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=9
    )

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Category (Aug 2021 - Jun 2022)")
plt.xticks(rotation=45)

#Legend follows color order for easier parsing
plt.legend(title="Category", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



[921]: #Again aligns close with GP appointments - makes sense given these are General Consultations

4 Appointments Regional Analysis

```
[922]: #See shape of the dataframe
AR.shape
```

```
[922]: (596821, 7)
```

```
[923]: #Mostly categorical data, only count of appts is int
print(AR.dtypes)
```

```
icb_ons_code           object
appointment_month      object
appointment_status     object
hcp_type               object
appointment_mode        object
time_between_book_and_appointment  object
count_of_appointments  int64
dtype: object
```

```
[924]: #No Null values
AR.isnull().sum()
```

```
[924]: icb_ons_code           0
appointment_month      0
appointment_status     0
hcp_type               0
appointment_mode        0
time_between_book_and_appointment  0
count_of_appointments  0
dtype: int64
```

```
[925]: #Over 20000 duplicate rows found
print("Duplicates:",AR.duplicated().sum())
```

```
Duplicates: 21604
```

```
[926]: #Creating AR2, a new df without duplicates
AR2 = AR.drop_duplicates()
print("Duplicates:",AR2.duplicated().sum())
```

```
Duplicates: 0
```

```
[927]: #AR2 reduced to 575,217 rows
AR2.shape
```

```
[927]: (575217, 7)
```

```
[928]: #View first 5 rows
AR2.head()
```

```
[928]:   icb_ons_code appointment_month appointment_status hcp_type appointment_mode \
0      E54000034      2020-01      Attended      GP      Face-to-Face
1      E54000034      2020-01      Attended      GP      Face-to-Face
2      E54000034      2020-01      Attended      GP      Face-to-Face
3      E54000034      2020-01      Attended      GP      Face-to-Face
4      E54000034      2020-01      Attended      GP      Face-to-Face

      time_between_book_and_appointment  count_of_appointments
0                                1 Day                        8107
1                   15 to 21 Days                        6791
2                   2 to 7 Days                        20686
3                   22 to 28 Days                        4268
4                   8 to 14 Days                        11971
```

```
[929]: #View last 5 rows - shows unknown values in appointment_status, hcp_type, and
      ↪ appointment_mode
AR2.tail()
```

```
[929]:   icb_ons_code appointment_month appointment_status hcp_type \
596813      E54000050      2022-06      Unknown      Unknown
596815      E54000050      2022-06      Unknown      Unknown
596816      E54000050      2022-06      Unknown      Unknown
596818      E54000050      2022-06      Unknown      Unknown
596819      E54000050      2022-06      Unknown      Unknown

      appointment_mode time_between_book_and_appointment \
596813      Telephone      Same Day
596815      Unknown      15 to 21 Days
596816      Unknown      2 to 7 Days
596818      Unknown      8 to 14 Days
596819      Unknown      More than 28 Days

      count_of_appointments
596813      10
596815      13
596816      21
596818      28
596819      17
```

```
[930]: #Only numerical shows
AR2.describe()
```

```
[930]:      count_of_appointments
count      575217.0000
```

```

mean          1290.8689
std           5960.8681
min            1.0000
25%            8.0000
50%           52.0000
75%          332.0000
max         211265.0000

```

```

[931]: #Rename columns to easier to use names
cols = AR2.columns.tolist()

cols[0] = "ICB_ONS"
cols[1] = "Appt_Month"
cols[2] = "Status"
cols[3] = "Type"
cols[4] = "Mode"
cols[5] = "Interval"
cols[6] = "#_Appts"

AR2.columns = cols
print(AR2)

```

	ICB_ONS	Appt_Month	Status	Type	Mode \
0	E54000034	2020-01	Attended	GP	Face-to-Face
1	E54000034	2020-01	Attended	GP	Face-to-Face
2	E54000034	2020-01	Attended	GP	Face-to-Face
3	E54000034	2020-01	Attended	GP	Face-to-Face
4	E54000034	2020-01	Attended	GP	Face-to-Face
...	...	...	...	...	...
596813	E54000050	2022-06	Unknown	Unknown	Telephone
596815	E54000050	2022-06	Unknown	Unknown	Unknown
596816	E54000050	2022-06	Unknown	Unknown	Unknown
596818	E54000050	2022-06	Unknown	Unknown	Unknown
596819	E54000050	2022-06	Unknown	Unknown	Unknown

	Interval	#_Appts
0	1 Day	8107
1	15 to 21 Days	6791
2	2 to 7 Days	20686
3	22 to 28 Days	4268
4	8 to 14 Days	11971
...	...	...
596813	Same Day	10
596815	15 to 21 Days	13
596816	2 to 7 Days	21
596818	8 to 14 Days	28
596819	More than 28 Days	17

[575217 rows x 7 columns]

```
[932]: #Change verified
print(AR2.columns)
```

```
Index(['ICB_ONS', 'Appt_Month', 'Status', 'Type', 'Mode', 'Interval',
       '#_Appts'],
      dtype='object')
```

```
[933]: #Top aggregation of appts is mix of face to face/telephone from two locations
↳(all same day)
AR2.sort_values(by="#_Appts", ascending=False).head()
```

```
[933]:
```

	ICB_ONS	Appt_Month	Status	Type	Mode	Interval	#_Appts
144544	E54000027	2020-01	Attended	GP	Face-to-Face	Same Day	211265
409005	E54000029	2020-01	Attended	GP	Face-to-Face	Same Day	203378
145905	E54000027	2020-06	Attended	GP	Telephone	Same Day	191019
410298	E54000029	2020-06	Attended	GP	Telephone	Same Day	186344
144828	E54000027	2020-02	Attended	GP	Face-to-Face	Same Day	185930

```
[934]: #In total 742,529,759 appts in AR (30 months)
#For comparison 296,046,770 in NC (11 months) and 167,980,692 in AD (7 months)
AR2["#_Appts"].sum()
```

```
[934]: 742529759
```

```
[935]: #Astounding number of Unknown value = not known if attended
AR2["Status"].value_counts().head()
```

```
[935]: Status
Attended    226295
Unknown     193374
DNA         155548
Name: count, dtype: int64
```

```
[936]: #Since Unknown Status accounts for 1/3 of data - will keep for the value it
↳provides
summary = pd.DataFrame({
    "Unknown_Count": [(AR2["Status"] == "Unknown").sum()],
    "Total": [len(AR2)]
})

summary["Unknown_%"] = summary["Unknown_Count"] / summary["Total"] * 100

summary
```

```
[936]:      Unknown_Count      Total      Unknown_%
0          193374    575217      33.6176
```

```
[937]: #Fairly consistent percentage across each ICB_ONS locations
summary_icb = (
    AR2.groupby("ICB_ONS")
        .agg(
            Unknown_Count=("Status", lambda x: (x == "Unknown").sum()),
            Total=("Status", "size")
        )
        .reset_index()
)

summary_icb["Unknown_%"] = (
    summary_icb["Unknown_Count"] / summary_icb["Total"] * 100
)

summary_icb = summary_icb.sort_values("Unknown_%", ascending=False)

summary_icb
```

```
[937]:      ICB_ONS      Unknown_Count      Total      Unknown_%
13  E54000028           2352      6579      35.7501
6   E54000019           2126      6013      35.3567
16  E54000031           2584      7324      35.2813
22  E54000039           2457      6993      35.1351
2   E54000011           2103      5996      35.0734
5   E54000018           2174      6303      34.4915
31  E54000052           2386      6921      34.4748
35  E54000056           2469      7178      34.3968
27  E54000044           6630     19300      34.3523
12  E54000027           2507      7299      34.3472
19  E54000036           2306      6714      34.3461
14  E54000029           2644      7700      34.3377
17  E54000032           2607      7615      34.2351
38  E54000059           1969      5759      34.1900
20  E54000037           2538      7453      34.0534
34  E54000055           2791      8206      34.0117
10  E54000025           5607     16498      33.9859
18  E54000034           1960      5768      33.9806
32  E54000053           6160     18130      33.9768
29  E54000050          13900     40925      33.9646
15  E54000030           2532      7462      33.9319
37  E54000058           2331      6875      33.9055
21  E54000038           2308      6813      33.8764
1   E54000010           8712     25731      33.8580
25  E54000042           4067     12021      33.8325
```


0	E54000008	15183	44884	33.8272
39	E54000060	3230	9591	33.6774
26	E54000043	2256	6707	33.6365
40	E54000061	7628	22695	33.6109
36	E54000057	16745	49975	33.5068
28	E54000048	11201	33477	33.4588
7	E54000022	2182	6582	33.1510
30	E54000051	9721	29431	33.0298
33	E54000054	8577	25978	33.0164
41	E54000062	2486	7561	32.8792
8	E54000023	5237	15939	32.8565
24	E54000041	1727	5345	32.3106
9	E54000024	1763	5467	32.2480
23	E54000040	1698	5272	32.2079
3	E54000013	1932	6061	31.8759
11	E54000026	6830	21485	31.7896
4	E54000015	4758	15191	31.3212

```
[938]: #A higher than average of Unknown occurred at beginning of pandemic March/April,
        ↪2020
        #https://www.instituteforgovernment.org.uk/sites/default/files/
        ↪timeline-lockdown-web.pdf
summary_month = (
    AR2.groupby("Appt_Month")
        .apply(
            lambda g: pd.Series({
                "Unknown_Appts": g.loc[g["Status"] == "Unknown", "#_Appts"].
                ↪sum(),
                "Total_Appts": g["#_Appts"].sum()
            })
        )
        .reset_index()
)

summary_month["Unknown_%"] = (
    summary_month["Unknown_Appts"] / summary_month["Total_Appts"] * 100
)

summary_month = summary_month.sort_values("Appt_Month")

summary_month
```

```
[938]:
```

	Appt_Month	Unknown_Appts	Total_Appts	Unknown_%
0	2020-01	1360095	27191758	5.0019
1	2020-02	1246047	24097066	5.1709
2	2020-03	2162530	24039635	8.9957
3	2020-04	1544368	16002488	9.6508

4	2020-05	1002869	16410591	6.1111
5	2020-06	1150826	20678416	5.5653
6	2020-07	1184058	22483207	5.2664
7	2020-08	1039751	20143878	5.1616
8	2020-09	1577116	26706462	5.9054
9	2020-10	1411617	28296051	4.9887
10	2020-11	965503	25056131	3.8534
11	2020-12	944052	23528063	4.0125
12	2021-01	943464	22482990	4.1963
13	2021-02	855988	22389461	3.8232
14	2021-03	968766	27216974	3.5594
15	2021-04	927521	23872938	3.8852
16	2021-05	847189	23488728	3.6068
17	2021-06	935938	26775653	3.4955
18	2021-07	945570	25728662	3.6752
19	2021-08	817933	23843177	3.4305
20	2021-09	1441525	28514685	5.0554
21	2021-10	1565438	30296850	5.1670
22	2021-11	1306088	30395923	4.2969
23	2021-12	1085511	25132174	4.3192
24	2022-01	958616	25623928	3.7411
25	2022-02	923334	25344812	3.6431
26	2022-03	1132719	29586020	3.8286
27	2022-04	915430	23904960	3.8295
28	2022-05	948878	27478652	3.4531
29	2022-06	941916	25819426	3.6481

[939]: *#The threshold isn't passed in this dataset - but it has the least amount of*
↪appointments of the three

[940]: *#Aligns with what is found in larger df, giving reassurance to the finding that*
↪6 months the threshold was passed
#Only month which is missing doesn't appear in the date range for this dataset

[942]: *#Back to exploration of AR df*
#Table showing percentage per category
mode_summary = (
 AR2.groupby("Mode")["#_Appts"]
 .sum()
 .reset_index()
)

mode_summary["Pct"] = (
 mode_summary["#_Appts"] /
 mode_summary["#_Appts"].sum() * 100
)

```
mode_summary.sort_values("#_Appts", ascending=False)
```

```
[942]:
```

	Mode	#_Appts	Pct
0	Face-to-Face	439876928	59.240310
2	Telephone	267752505	36.059498
3	Unknown	26439308	3.560707
1	Home Visit	4848743	0.653003
4	Video/Online	3612275	0.486482

```
[943]: #Most of the time GP appts are attended but its less likely when it's not -  
↳also documentation of  
#"Unknown" increases as other staff are used  
type_status = (  
    AR2.groupby(["Type", "Status"])["_Appts"]  
        .sum()  
        .unstack(fill_value=0)  
)  
  
type_status  
  
type_status_pct = (  
    type_status.div(type_status.sum(axis=1), axis=0) * 100  
)  
  
type_status_pct
```

```
[943]:
```

Status	Attended	DNA	Unknown
Type			
GP	94.272153	2.476183	3.251665
Other Practice staff	88.089365	6.122949	5.787686
Unknown	88.475738	2.752272	8.771990

```
[944]: #Now that other values are observed, we can remove everything else besides "GP"  
↳b/c the question  
#is specifically about "when patients miss general practitioner (GP)"  
↳appointments"  
AR2 = AR2[AR2["Type"] == "GP"].copy()
```

```
[945]: #Check if worked  
AR2["Type"].value_counts()
```

```
[945]:
```

Type	
GP	217828

Name: count, dtype: int64

```
[1000]: AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])
```

```

#Total appointments per month across all ICBs
monthly_totals = (
    AR2.groupby("Appt_Month")["#_Appts"]
        .sum()
        .reset_index()
)

#Full Capacity Working days (Mon-Fri)
monthly_totals["Working_Days"] = monthly_totals["Appt_Month"].apply(
    lambda d: np.busday_count(
        d.date(),
        (d + pd.offsets.MonthBegin(1)).date()
    )
)

#Weekend Calculation (~2.09% total activity from NC data)
#Saturday = 1.883% + Sunday = 0.207% 2.0905%
weekend_uplift = 1.020905

monthly_totals["Adjusted_Working_Days"] = (
    monthly_totals["Working_Days"] * weekend_uplift
)

# Average appointments per adjusted working day
monthly_totals["Avg_Appts_Per_Working_Day"] = (
    monthly_totals["#_Appts"] / monthly_totals["Adjusted_Working_Days"]
)

# Threshold
threshold = 1_200_000

monthly_totals["Above_1.2M"] = (
    monthly_totals["Avg_Appts_Per_Working_Day"] > threshold
)

# Prevent scientific notation
pd.set_option("display.float_format", "{:,.0f}".format)

# Output
print("Monthly appointments (adjusted for small weekend activity uplift):")
display(monthly_totals)

print("\nMonths above threshold:")
display(monthly_totals[monthly_totals["Above_1.2M"]])

peak_month = monthly_totals.loc[
    monthly_totals["Avg_Appts_Per_Working_Day"].idxmax()
]

```

```
]
print("\nPeak month:")
display(peak_month.to_frame().T)
```

Monthly appointments (adjusted for small weekend activity uplift):

	Appt_Month	#_Appts	Working_Days	Adjusted_Working_Days	\
0	2020-01-01	14166016	23	23	
1	2020-02-01	12465443	20	20	
2	2020-03-01	12833348	22	22	
3	2020-04-01	8794204	22	22	
4	2020-05-01	8921701	21	21	
5	2020-06-01	11046306	22	22	
6	2020-07-01	11698711	23	23	
7	2020-08-01	10479258	21	21	
8	2020-09-01	13185055	22	22	
9	2020-10-01	13180551	22	22	
10	2020-11-01	12499729	21	21	
11	2020-12-01	11757356	23	23	
12	2021-01-01	11803633	21	21	
13	2021-02-01	11824930	20	20	
14	2021-03-01	14386905	23	23	
15	2021-04-01	12461199	22	22	
16	2021-05-01	12320186	21	21	
17	2021-06-01	14046347	22	22	
18	2021-07-01	13333534	22	22	
19	2021-08-01	12296193	22	22	
20	2021-09-01	14484242	22	22	
21	2021-10-01	14298054	21	21	
22	2021-11-01	14893177	22	22	
23	2021-12-01	12651618	23	23	
24	2022-01-01	13131239	21	21	
25	2022-02-01	12866186	20	20	
26	2022-03-01	15122640	23	23	
27	2022-04-01	11906042	21	21	
28	2022-05-01	13788824	22	22	
29	2022-06-01	12889429	22	22	

	Avg_Appts_Per_Working_Day	Above_1.2M
0	603,302	False
1	610,509	False
2	571,389	False
3	391,551	False
4	416,143	False
5	491,823	False
6	498,224	False
7	488,794	False

8	587,048	False
9	586,848	False
10	583,037	False
11	500,722	False
12	550,568	False
13	579,140	False
14	612,709	False
15	554,820	False
16	574,662	False
17	625,396	False
18	593,659	False
19	547,473	False
20	644,893	False
21	666,918	False
22	663,100	False
23	538,807	False
24	612,493	False
25	630,136	False
26	644,042	False
27	555,345	False
28	613,931	False
29	573,886	False

Months above threshold:

Empty DataFrame

Columns: [Appt_Month, #_Appts, Working_Days, Adjusted_Working_Days, Avg_Appts_Per_Working_Day, Above_1.2M]

Index: []

Peak month:

	Appt_Month	#_Appts	Working_Days	Adjusted_Working_Days	\
21	2021-10-01 00:00:00	14298054	21	21	

	Avg_Appts_Per_Working_Day	Above_1.2M
21	666,918	False

```
[1006]: #Visualization of month-by-month capacity
plot_df = monthly_totals.sort_values("Appt_Month")

below = plot_df[plot_df["Avg_Appts_Per_Working_Day"] <= threshold]
above = plot_df[plot_df["Avg_Appts_Per_Working_Day"] > threshold]

plt.figure(figsize=(14, 6))

#Line
```

```

plt.plot(
    plot_df["Appt_Month"],
    plot_df["Avg_Appts_Per_Working_Day"],
    color="steelblue"
)

#Points below threshold
plt.scatter(
    below["Appt_Month"],
    below["Avg_Appts_Per_Working_Day"],
    color="steelblue"
)

#Threshold line
plt.axhline(
    y=threshold,
    color="red",
    linestyle="--",
    label="Threshold (1,200,000)"
)

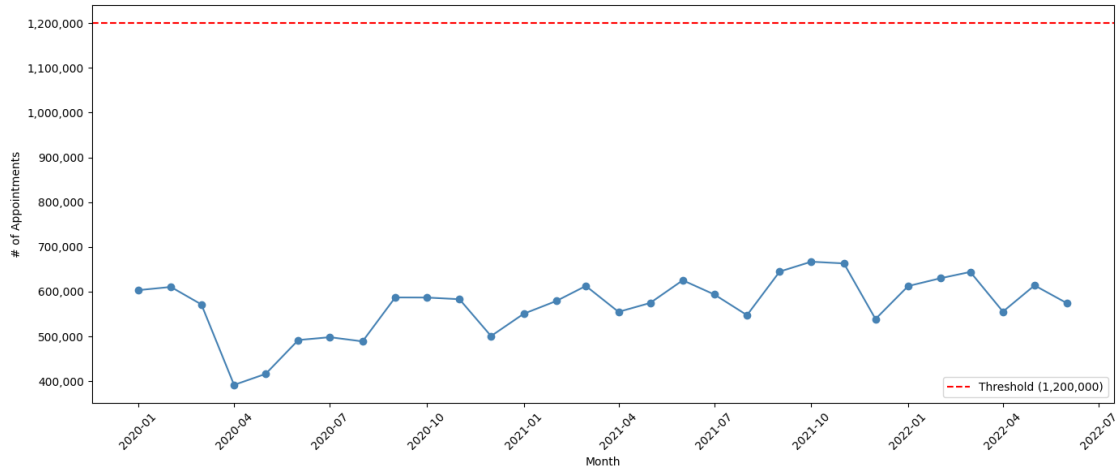
#Labels for points above threshold
for _, row in above.iterrows():
    plt.annotate(
        row["Appt_Month"].strftime("%b %Y"),
        (row["Appt_Month"], row["Avg_Appts_Per_Working_Day"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=9,
        fontweight="bold"
    )

plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter("{x:,.0f}")
)

plt.xlabel("Month")
plt.ylabel("# of Appointments")
plt.xticks(rotation=45)

plt.legend()
plt.tight_layout()
plt.show()

```



```
[948]: #Same calculation but with the Actual Duration dataset
AD["Appt_Date"] = pd.to_datetime(AD["Appt_Date"])

AD["Appt_Month"] = AD["Appt_Date"].dt.to_period("M").dt.to_timestamp()

monthly_totals = (
    AD.groupby("Appt_Month")["Appts"]
      .sum()
      .reset_index()
)

monthly_totals["Working_Days"] = monthly_totals["Appt_Month"].apply(
    lambda d: np.busday_count(
        d.date(),
        (d + pd.offsets.MonthBegin(1)).date()
    )
)

weekend_uplift = 1.020905

monthly_totals["Adjusted_Working_Days"] = (
    monthly_totals["Working_Days"] * weekend_uplift
)

monthly_totals["Avg_Appts_Per_Working_Day"] = (
    monthly_totals["Appts"]
    / monthly_totals["Adjusted_Working_Days"]
)

threshold = 1_200_000
```



```

monthly_totals["Above_1.2M"] = (
    monthly_totals["Avg_Appts_Per_Working_Day"] > threshold
)

pd.set_option("display.float_format", "{:,.0f}".format)

print("Monthly appointments (adjusted for weekend activity):")
display(monthly_totals)

print("\nMonths above threshold:")
display(
    monthly_totals[
        monthly_totals["Above_1.2M"]
    ]
)

peak_month = monthly_totals.loc[
    monthly_totals["Avg_Appts_Per_Working_Day"].idxmax()
]

print("\nPeak month:")
display(peak_month.to_frame().T)

```

Monthly appointments (adjusted for weekend activity):

	Appt_Month	Appts	Working_Days	Adjusted_Working_Days	\
0	2021-12-01	22853483	23	23	
1	2022-01-01	23597196	21	21	
2	2022-02-01	23351939	20	20	
3	2022-03-01	27170002	23	23	
4	2022-04-01	21948814	21	21	
5	2022-05-01	25343941	22	22	
6	2022-06-01	23715317	22	22	

	Avg_Appts_Per_Working_Day	Above_1.2M
0	973,283	False
1	1,100,667	False
2	1,143,688	False
3	1,157,115	False
4	1,023,780	False
5	1,128,408	False
6	1,055,895	False

Months above threshold:

Empty DataFrame

```
Columns: [Appt_Month, Appts, Working_Days, Adjusted_Working_Days,
↪Avg_Appts_Per_Working_Day, Above_1.2M]
Index: []
```

Peak month:

	Appt_Month	Appts	Working_Days	Adjusted_Working_Days	\
3	2022-03-01 00:00:00	27170002	23	23	

	Avg_Appts_Per_Working_Day	Above_1.2M
3	1,157,115	False

```
[1007]: #Filter to just GP
df = AR2[
    (AR2["Mode"].isin(["Face-to-Face", "Home Visit", "Telephone", "Video/
↪Online", "Unknown"]))
].copy()

summary = (
    df.groupby("Mode")["_Appts"]
      .sum()
      .reset_index()
)

summary = summary.sort_values("#_Appts", ascending=False)

plt.figure(figsize=(8, 5))

ax = sns.barplot(
    data=summary,
    x="Mode",
    y="#_Appts",
    palette=["steelblue", "orange", "crimson", "purple", "gray"] #different_
↪colours
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}'))
)

#Add value labels
for p in ax.patches:
    ax.annotate(
        f"{int(p.get_height()):,}",
        (p.get_x() + p.get_width() / 2, p.get_height()),
```

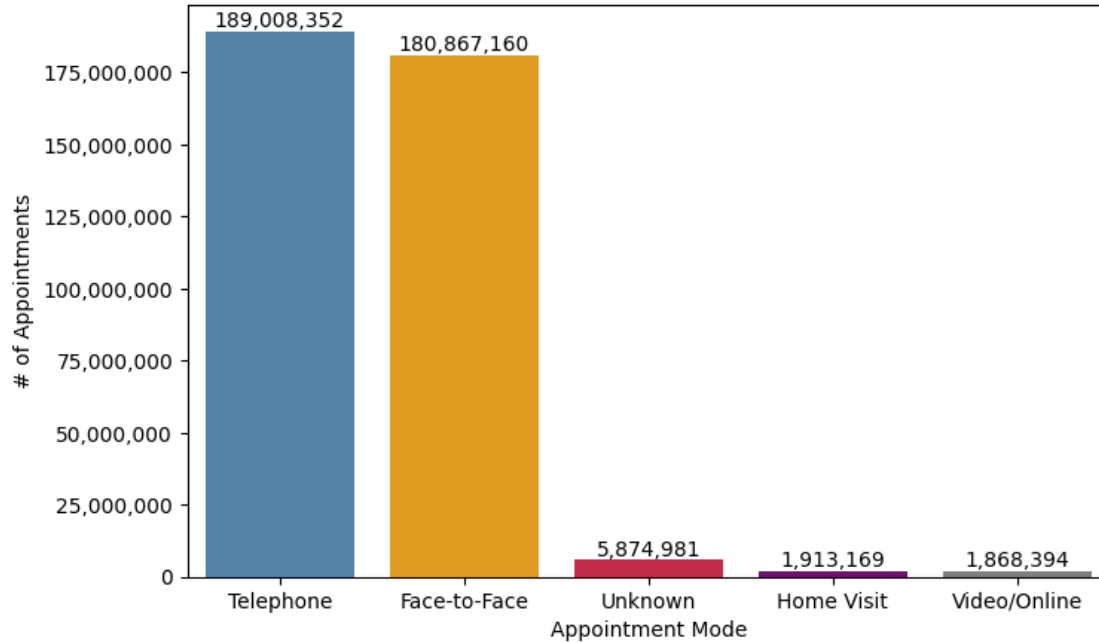
```

        ha="center",
        va="bottom"
    )

plt.xlabel("Appointment Mode")
plt.ylabel("# of Appointments")

```

```
[1007]: Text(0, 0.5, '# of Appointments')
```



```
[950]: #Face-to-Face is most prevalent x2 of Telephone
       #Other types are fairly fringe
```

```
[951]: #Only mapping three of the most relevant Modes (Face to Face, Telephone, Video)
AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30") &
    (AR2["Mode"].isin(["Face-to-Face", "Telephone", "Video/Online"]))
].copy()

monthly_type = (
    df.set_index("Appt_Month")
      .groupby("Mode")["_Appts"]
      .resample("MS")

```

```

        .sum()
        .reset_index()
    )

mode_order = (
    monthly_type.groupby("Mode")["#_Appts"]
    .sum()
    .sort_values(ascending=False)
    .index
    .tolist()
)

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly_type,
    x="Appt_Month",
    y="#_Appts",
    hue="Mode",
    hue_order=mode_order,
    marker="o"
)

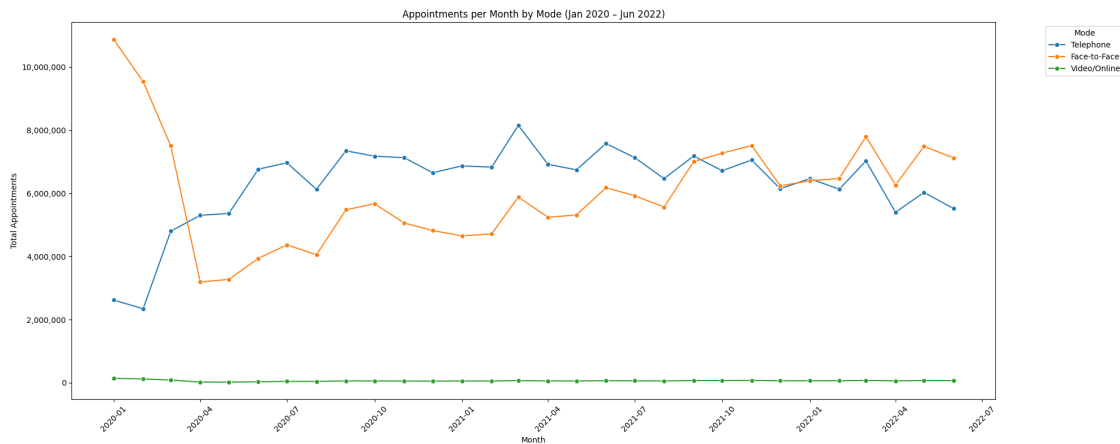
ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Mode (Jan 2020 - Jun 2022)")
plt.xticks(rotation=45)

plt.legend(title="Mode", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



[952]: *#Telephone appointments recently become more standard big jump beginning of 2020 and gradual increase till plateau which has maintain*

```
[953]: #Exploring status
df = AR2[
    AR2["Status"].isin(["Attended", "DNA", "Unknown"])
].copy()

summary = (
    df.groupby("Status")["_Appts"]
      .sum()
      .reset_index()
)

summary = summary.sort_values("#_Appts", ascending=False)

plt.figure(figsize=(8, 5))

ax = sns.barplot(
    data=summary,
    x="Status",
    y="#_Appts",
    palette=["steelblue", "orange", "crimson"]
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

for p in ax.patches:
```

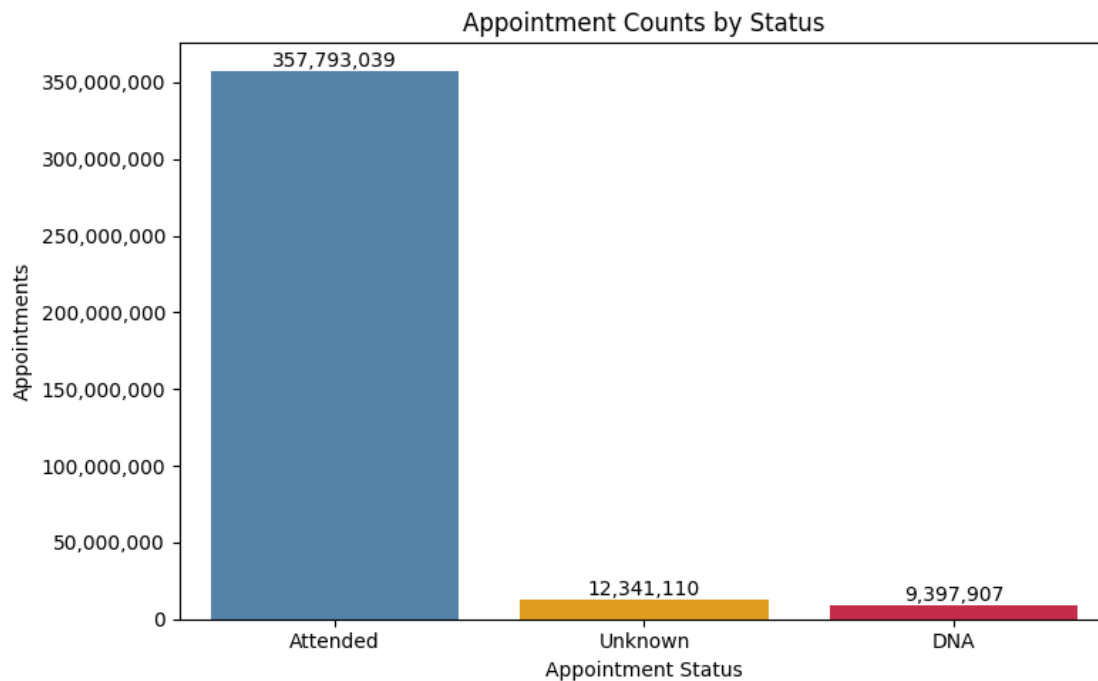
```

ax.annotate(
    f"{p.get_height():,.0f}",
    (p.get_x() + p.get_width() / 2, p.get_height()),
    ha="center",
    va="bottom",
    fontsize=10
)

plt.xlabel("Appointment Status")
plt.ylabel("Appointments")
plt.title("Appointment Counts by Status")

plt.tight_layout()
plt.show()

```



```

[954]: #Checking how this looks over time
AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30")
].copy()

monthly_status = (
    df.set_index("Appt_Month")

```

```

        .groupby("Status")["#_Appts"]
        .resample("MS")
        .sum()
        .reset_index()
    )

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly_status,
    x="Appt_Month",
    y="#_Appts",
    hue="Status",
    marker="o"
)

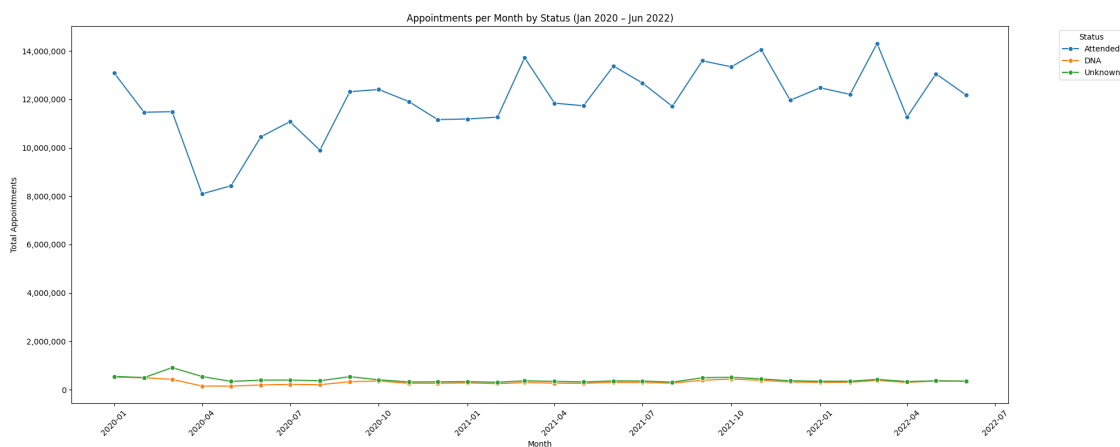
ax.yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Status (Jan 2020 - Jun 2022)")

plt.xticks(rotation=45)
plt.legend(title="Status", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



```

[955]: #Counting types
AR2["Type"].value_counts().head(10)

```

```
[955]: Type
GP      217828
Name: count, dtype: int64
```

```
[956]: #Create a group with interval/status + appt count then sum
type_status = (
    AR2.groupby(["Type", "Status"])["#_Appts"]
        .sum()
        .unstack(fill_value=0)
)

type_status

#Make the values into percentages
type_status_pct = (
    type_status.div(type_status.sum(axis=1), axis=0) * 100
)

type_status_pct

#Appears to be linear
```

```
[956]: Status    Attended    DNA    Unknown
Type
GP              94         2         3
```

```
[958]: #Only twice in full date range does other staff get called upon more than a GP
#October 2020 / 2021 both of which were times when capacity was overreached..
#Likely people want to see their GP but couldn't
AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30")
].copy()

monthly_type = (
    df.set_index("Appt_Month")
        .groupby("Type")["#_Appts"]
        .resample("MS")
        .sum()
        .reset_index()
)

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
```



```

data=monthly_type,
x="Appt_Month",
y="#_Appts",
hue="Type",
marker="o"
)

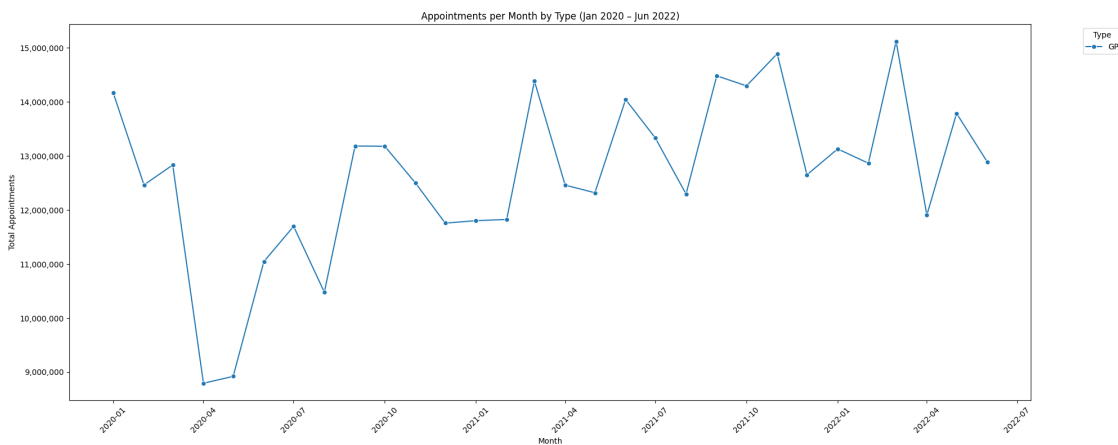
ax.yaxis.set_major_formatter(ticker.StrMethodFormatter('{x:,.0f}'))

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Type (Jan 2020 - Jun 2022)")

plt.xticks(rotation=45)
plt.legend(title="Type", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



```

[959]: #View count of each Interval value
AR2["Interval"].value_counts().head(10)

```

```

[959]: Interval
Same Day                34811
2 to 7 Days             33638
1 Day                   32340
8 to 14 Days            30211
15 to 21 Days           27127
22 to 28 Days           25272
More than 28 Days       23660
Unknown / Data Quality  10769
Name: count, dtype: int64

```

```
[960]: #Create a group with interval/status + appt count then sum
Interval_status = (
    AR2.groupby(["Interval", "Status"])["_Appts"]
        .sum()
        .unstack(fill_value=0)
)

Interval_status

#Make the values into percentages
Interval_status_pct = (
    Interval_status.div(Interval_status.sum(axis=1), axis=0) * 100
)

Interval_status_pct

#Appears to be linear
```

```
[960]: Status                Attended  DNA  Unknown
Interval
1 Day                94    3      3
15 to 21 Days       87    5      8
2 to 7 Days         93    4      4
22 to 28 Days       84    5     10
8 to 14 Days        90    5      6
More than 28 Days   73    7     20
Same Day            97    1      2
Unknown / Data Quality 73    3     24
```

```
[961]: #Exploring visually the Interval Count
interval_order = [
    "Same Day",
    "1 Day",
    "2 to 7 Days",
    "8 to 14 Days",
    "15 to 21 Days",
    "22 to 28 Days",
    "More than 28 Days",
    "Unknown / Data Quality"
]

df = AR2[
    AR2["Interval"].isin(interval_order)
].copy()

summary = (
    df.groupby("Interval")["_Appts"]
```

```

        .sum()
        .reset_index()
    )

summary = summary.sort_values("#_Appts", ascending=False)

palette = [
    "steelblue",
    "orange",
    "crimson",
    "seagreen",
    "purple",
    "darkcyan",
    "gold",
    "gray"
]

plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data=summary,
    x="Interval",
    y="#_Appts",
    palette=palette[:len(summary)]
)

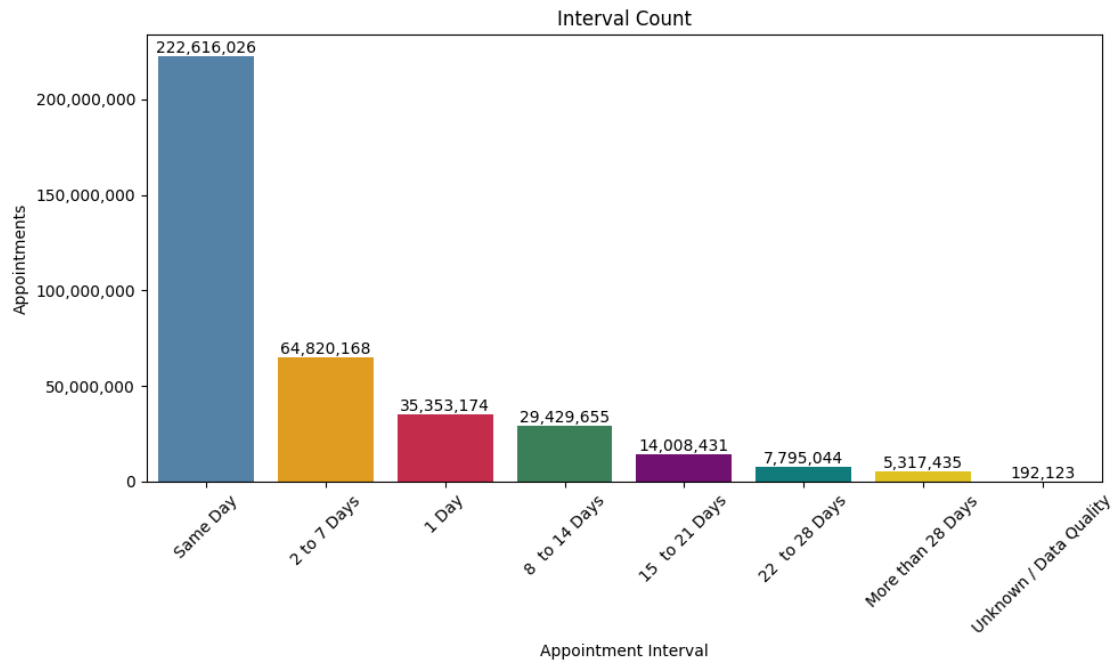
ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

for p in ax.patches:
    ax.annotate(
        f"{p.get_height():,.0f}",
        (p.get_x() + p.get_width() / 2, p.get_height()),
        ha="center",
        va="bottom",
        fontsize=10
    )

plt.xlabel("Appointment Interval")
plt.ylabel("Appointments")
plt.title("Interval Count")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

```



[962]: *#Creating Generalized Linear Regression Model with Interval as the predictor*
#GP appointments only

```
#Map intervals
interval_order = [
    "Same Day",
    "1 Day",
    "2 to 7 Days",
    "8 to 14 Days",
    "15 to 21 Days",
    "22 to 28 Days",
    "More than 28 Days"
]

mapping = {v: i for i, v in enumerate(interval_order)}

# Filter to GP appointments and valid intervals
df = AR2[
    (AR2["Type"] == "GP") &
    (AR2["Interval"].isin(interval_order))
].copy()

df["Interval_Num"] = df["Interval"].map(mapping)
```

```

df["Attended"] = (df["Status"] == "Attended").astype(int)

#Logistic regression
model = smf.glm(
    formula="Attended ~ Interval_Num",
    data=df,
    family=sm.families.Binomial(),
    freq_weights=df["_Appts"]
).fit()

print(model.summary())

# Odds ratio interpretation
print("\nOdds Ratio:")
print(np.exp(model.params))

#Attendance probability declined steadily as waiting time increased, suggesting
↳that longer delays
#between booking and appointment are associated with lower attendance rates for
↳GP consultations.

```

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          Attended    No. Observations:          207059
Model:                  GLM         Df Residuals:              379339931
Model Family:           Binomial    Df Model:                  1
Link Function:           Logit      Scale:                    1.0000
Method:                 IRLS        Log-Likelihood:           -7.8954e+07
Date:                   Mon, 25 May 2026    Deviance:                 1.5791e+08
Time:                   23:53:50    Pearson chi2:             3.77e+08
No. Iterations:          9          Pseudo R-squ. (CS):       1.000
Covariance Type:        nonrobust
=====

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	3.3230	0.000	1.05e+04	0.000	3.322	3.324
Interval_Num	-0.3711	0.000	-3037.234	0.000	-0.371	-0.371

```

Odds Ratio:
Intercept      28
Interval_Num    1
dtype: float64

```

```

[963]: interval_order = [
        "Same Day",

```

```

    "1 Day",
    "2 to 7 Days",
    "8 to 14 Days",
    "15 to 21 Days",
    "22 to 28 Days",
    "More than 28 Days"
]

#GP appointments only
df = AR2[
    (AR2["Type"] == "GP") &
    (AR2["Interval"].isin(interval_order))
].copy()

mapping = {v: i for i, v in enumerate(interval_order)}
df["Interval_Num"] = df["Interval"].map(mapping)

#Binary outcome
df["Attended"] = (df["Status"] == "Attended").astype(int)

#Logistic regression
model = smf.glm(
    formula="Attended ~ Interval_Num",
    data=df,
    family=sm.families.Binomial(),
    freq_weights=df["_Appts"]
).fit()

#Weighted observed attendance rates
obs = (
    df.groupby("Interval")
    .apply(
        lambda x: np.average(
            x["Attended"],
            weights=x["_Appts"]
        ),
        include_groups=False
    )
    .reindex(interval_order)
)

pred_df = pd.DataFrame({
    "Interval_Num": range(len(interval_order)),
    "Interval": interval_order
})

pred_df["Predicted"] = model.predict(pred_df)

```

```

# Plot
plt.figure(figsize=(10, 5))

x = np.arange(len(interval_order))

plt.plot(
    x,
    obs.values,
    marker="o",
    linewidth=2,
    color="steelblue",
    label="Observed"
)

plt.plot(
    x,
    pred_df["Predicted"],
    marker="o",
    linewidth=2,
    color="darkorange",
    label="Predicted"
)

for i in range(len(interval_order)):

    plt.annotate(
        f"{obs.iloc[i]:.1%}",
        (x[i], obs.iloc[i]),
        textcoords="offset points",
        xytext=(0, 8),
        ha="center",
        color="steelblue",
        fontsize=8
    )

    plt.annotate(
        f"{pred_df['Predicted'].iloc[i]:.1%}",
        (x[i], pred_df["Predicted"].iloc[i]),
        textcoords="offset points",
        xytext=(0, -14),
        ha="center",
        color="darkorange",
        fontsize=8
    )

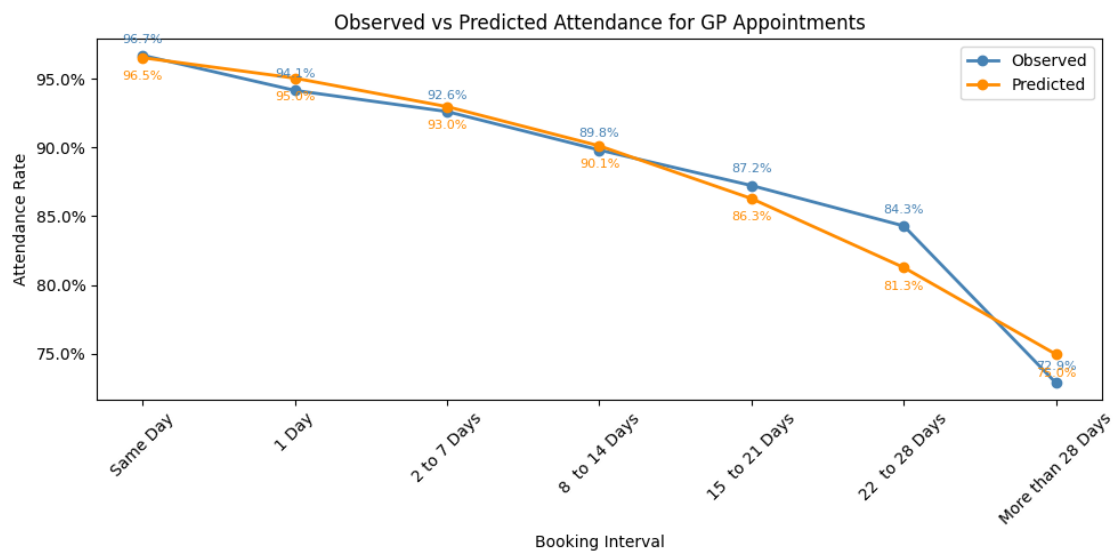
plt.gca().yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

```

```
plt.xticks(x, interval_order, rotation=45)
plt.ylabel("Attendance Rate")
plt.xlabel("Booking Interval")
plt.title("Observed vs Predicted Attendance for GP Appointments")

plt.legend()
plt.tight_layout()
plt.show()

# The predicted curve closely follows the observed attendance rates,
# suggesting booking interval alone explains a substantial proportion
# of the variation in attendance for GP appointments.
```



```
[964]: AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30")
].copy()

monthly_type = (
    df.set_index("Appt_Month")
      .groupby("Interval")["#_Appts"]
      .resample("MS")
      .sum()
      .reset_index()
)
```



```

interval_order = (
    monthly_type.groupby("Interval")["#_Appts"]
    .sum()
    .sort_values(ascending=False)
    .index
    .tolist()
)

palette = [
    "steelblue",
    "orange",
    "crimson",
    "seagreen",
    "purple",
    "darkcyan",
    "gold",
    "gray"
]

plt.figure(figsize=(24, 12))

ax = sns.lineplot(
    data=monthly_type,
    x="Appt_Month",
    y="#_Appts",
    hue="Interval",
    hue_order=interval_order,
    palette=palette[:len(interval_order)],
    marker="o"
)

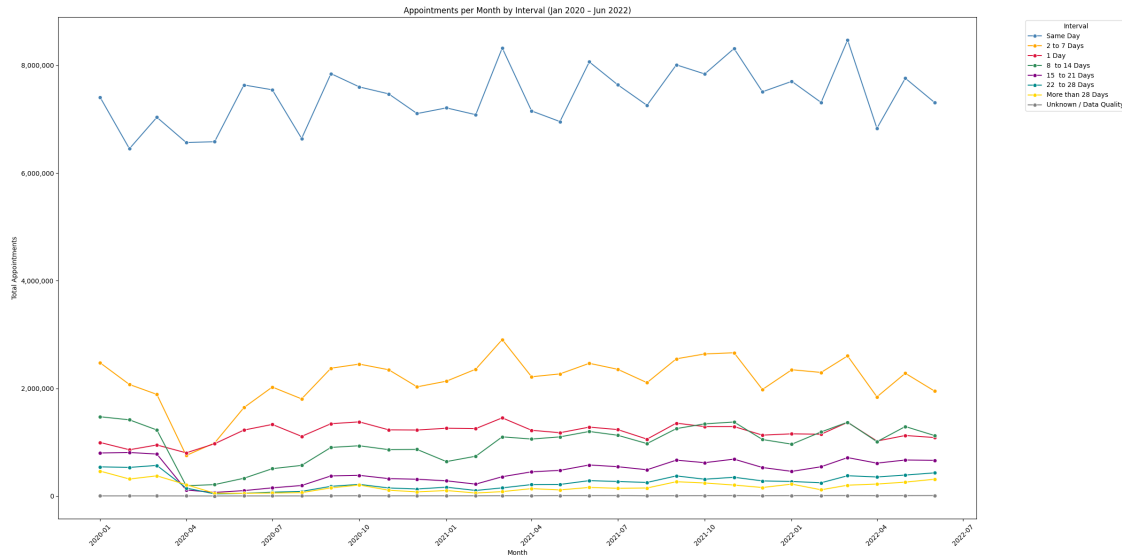
ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Appointments per Month by Interval (Jan 2020 - Jun 2022)")
plt.xticks(rotation=45)

plt.legend(title="Interval", bbox_to_anchor=(1.05, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



[965]: *#Same day appointments are by far the most common*

```
[966]: plt.figure(figsize=(12, 6))

x_vals = attended_pct["Interval"].astype(str)
y_vals = attended_pct["Pct_Attended"]

#Define color mapping by category
color_map = {
    "Same Day": "green",
    "1 Day": "green",
    "2 to 7 Days": "green",
    "8 to 14 Days": "green",
    "15 to 21 Days": "gold",
    "22 to 28 Days": "gold",
    "More than 28 Days": "red"
}

plt.plot(
    x_vals,
    y_vals,
    color="steelblue",
    marker="o"
)

for x, y in zip(x_vals, y_vals):
    plt.scatter(
```

```

        x,
        y,
        color=color_map.get(x, "gray"),
        s=90,
        zorder=3
    )

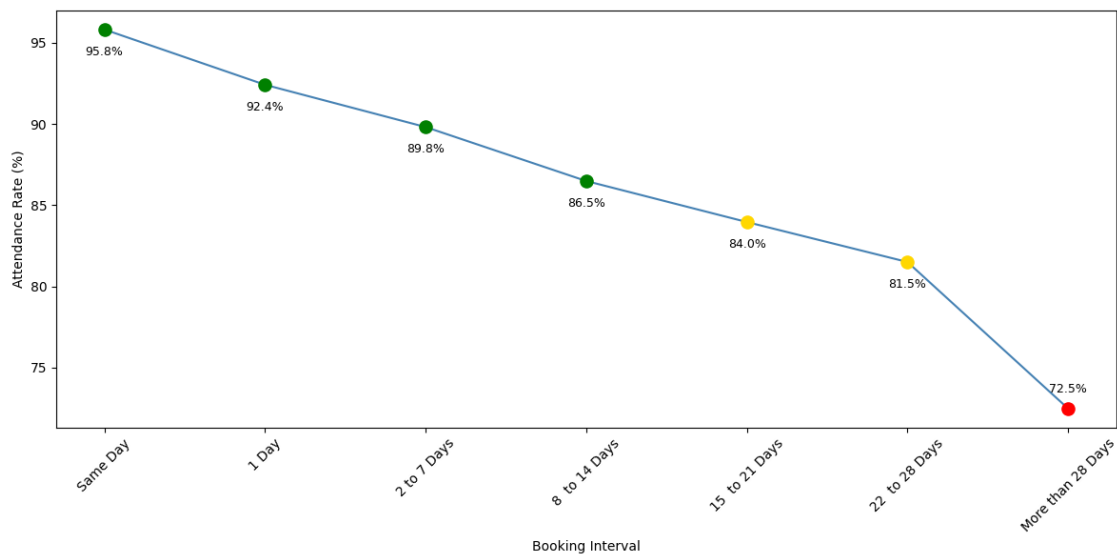
    #label positioning fix (last one higher)
    offset = -12
    if x == "More than 28 Days":
        offset = 10

    plt.annotate(
        f"{y:.1f}%",
        (x, y),
        textcoords="offset points",
        xytext=(0, offset),
        ha="center",
        va="top" if offset < 0 else "bottom",
        fontsize=9
    )

plt.xlabel("Booking Interval")
plt.ylabel("Attendance Rate (%)")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```

[967]: AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30")
].copy()

def assign_season(date):
    m = date.month
    y = date.year

    if m in [6, 7, 8]:
        return f"Summer {y}"
    elif m in [9, 10, 11]:
        return f"Autumn {y}"
    elif m in [12]:
        return f"Winter {y}/{y+1}"
    elif m in [1, 2]:
        return f"Winter {y-1}/{y}"
    else: # Mar-May
        return f"Spring {y}"

df["Season"] = df["Appt_Month"].apply(assign_season)

# Aggregate
season_appts = (
    df.groupby("Season")["#_Appts"]
      .sum()
      .reset_index()
)

# Proper chronological ordering
season_order = sorted(
    season_appts["Season"].unique(),
    key=lambda x: (
        int(x.split()[1].split("/")[0]), # base year
        ["Summer", "Autumn", "Winter", "Spring"].index(x.split()[0])
    )
)

season_appts["Season"] = pd.Categorical(
    season_appts["Season"],
    categories=season_order,
    ordered=True
)

season_appts = season_appts.sort_values("Season").reset_index(drop=True)

```

```

x = season_appts["Season"].astype(str)
y = season_appts["_Appts"]

colors = ["steelblue", "orange", "crimson", "seagreen"]

plt.figure(figsize=(12, 6))

# Line
plt.plot(x, y, color="steelblue", marker="o")

# Points + labels
for i in range(len(season_appts)):
    plt.scatter(x[i], y[i], color=colors[i % len(colors)], s=100, zorder=3)

    plt.text(
        x[i],
        y[i],
        f"{y[i]:.0f}",
        ha="center",
        va="bottom",
        fontsize=10
    )

#change circles
pct_change = season_appts["_Appts"].pct_change() * 100

for i in range(1, len(season_appts)):
    x_pos = i - 0.5
    y_pos = (y[i] + y[i - 1]) / 2

    dx = -0.25 if i == 1 else 0.25

    plt.scatter(
        x_pos + dx,
        y_pos,
        s=1200,
        facecolors="none",
        edgecolors="black",
        linewidths=1.2,
        zorder=2
    )

    plt.text(
        x_pos + dx,
        y_pos,
        f"{pct_change[i]:+.1f}%",

```

```

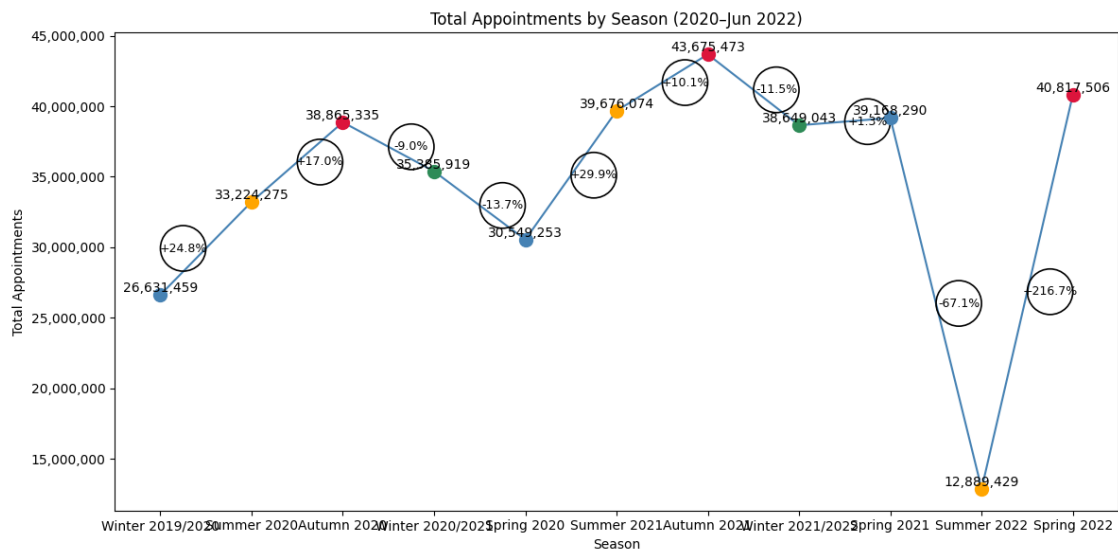
        ha="center",
        va="center",
        fontsize=9
    )

plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.xlabel("Season")
plt.ylabel("Total Appointments")
plt.title("Total Appointments by Season (2020-Jun 2022)")

plt.tight_layout()
plt.show()

```



```
[968]: AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])
```

```

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-05-31")
].copy()

monthly_appts = (
    df.groupby("Appt_Month")["#_Appts"]
      .sum()
      .reset_index()
      .sort_values("Appt_Month")
)

```

```

)

colors = [
    "steelblue",
    "orange",
    "crimson",
    "seagreen",
    "purple",
    "darkcyan",
    "gold",
    "gray",
    "steelblue",
    "orange",
    "crimson"
]

plt.figure(figsize=(14, 6))

bars = plt.bar(
    monthly_appts["Appt_Month"].dt.strftime("%b %Y"),
    monthly_appts["#_Appts"],
    color=colors[:len(monthly_appts)]
)

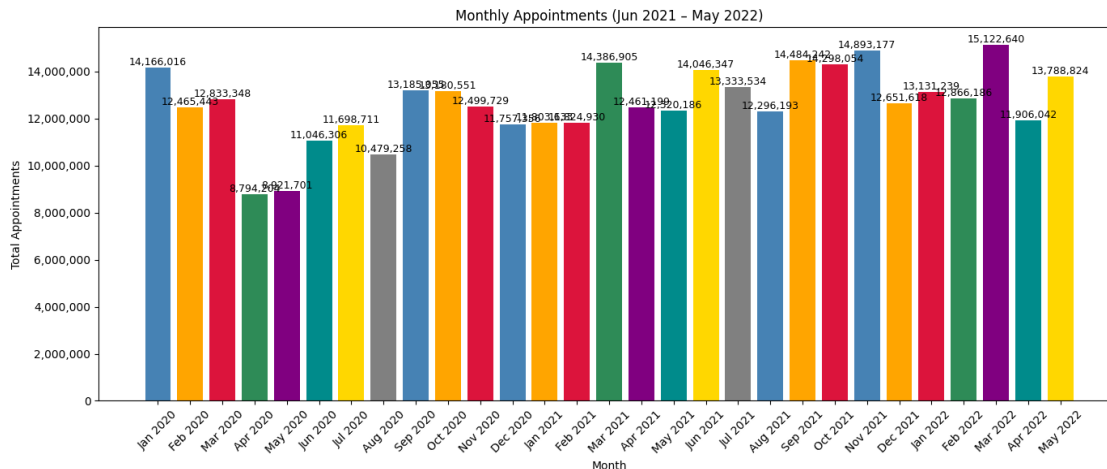
for bar in bars:
    y = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        y,
        f"{y:,.0f}",
        ha="center",
        va="bottom",
        fontsize=9
    )

plt.gca().yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}')
)

plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Monthly Appointments (Jun 2021 - May 2022)")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```
[969]: AR2["Appt_Month"] = pd.to_datetime(AR2["Appt_Month"])

df = AR2[
    (AR2["Appt_Month"] >= "2020-01-01") &
    (AR2["Appt_Month"] <= "2022-06-30")
].copy()

top_icb = (
    df.groupby("ICB_ONS")["#_Appts"]
      .sum()
      .nlargest(5)
      .index
)

df = df[df["ICB_ONS"].isin(top_icb)]

monthly = (
    df.groupby(
        ["ICB_ONS", pd.Grouper(key="Appt_Month", freq="MS")]
    )["#_Appts"]
      .sum()
      .reset_index()
)

plt.figure(figsize=(20, 8))

ax = sns.lineplot(
    data=monthly,
    x="Appt_Month",
    y="#_Appts",
    hue="ICB_ONS",
```



```

hue_order=top_icb,
marker="o"
)

ax.yaxis.set_major_formatter(
    ticker.StrMethodFormatter('{x:,.0f}'))
)

for _, row in monthly.iterrows():
    ax.annotate(
        f"{row['#_Appts']:,.0f}",
        (row["Appt_Month"], row["#_Appts"]),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=7
    )

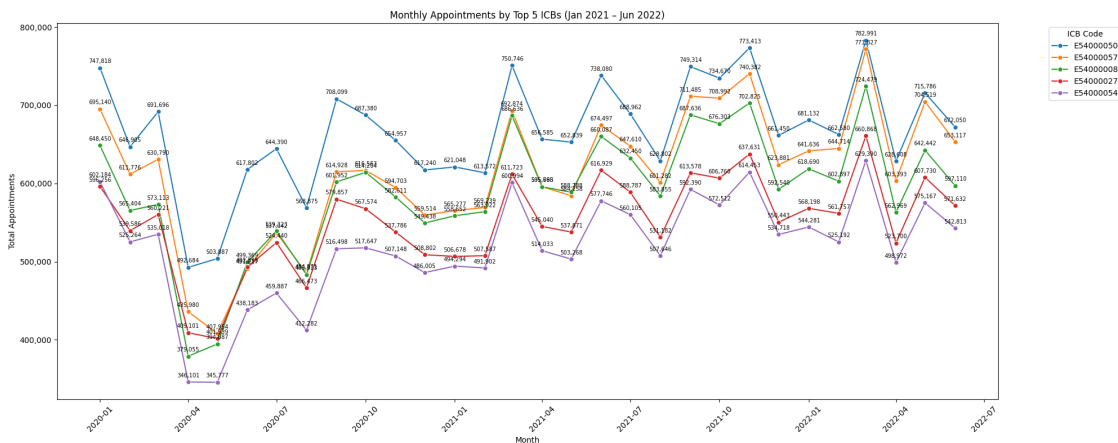
plt.xlabel("Month")
plt.ylabel("Total Appointments")
plt.title("Monthly Appointments by Top 5 ICBs (Jan 2021 - Jun 2022)")

plt.xticks(rotation=45)

plt.legend(
    title="ICB Code",
    bbox_to_anchor=(1.05, 1),
    loc="upper left"
)

plt.tight_layout()
plt.show()

```



5 Twitter Analysis

```
[970]: # View the DataFrame shape
TW.shape
```

```
[970]: (1174, 10)
```

```
[971]: #View first 5 rows
TW.head()
```

```
[971]:          tweet_id          tweet_full_text \
0  1567629223795527681  As Arkansas' first Comprehensive Stroke Certif...
1  1567582846612553728  RT @AndreaGrammer: Work-life balance is at the...
2  1567582787070304256  RT @OntarioGreens: $10 billion can go a long w...
3  1567582767625428992  RT @modrnhealthcr: #NEW: Insurance companies...
4  1567582720460570625  ICYMI: Our recent blogs on Cybersecurity in Ac...
```

```
          tweet_entities \
0  {'hashtags': [{'text': 'Healthcare', 'indices'...
1  {'hashtags': [{'text': 'PremiseHealth', 'indic...
2  {'hashtags': [{'text': 'Healthcare', 'indices'...
3  {'hashtags': [{'text': 'NEW', 'indices': [20, ...
4  {'hashtags': [{'text': 'blogs', 'indices': [18...
```

```
          tweet_entities_hashtags \
0          #Healthcare
1  #PremiseHealth, #hiring
2          #Healthcare
3          #NEW
4  #blogs, #digitaltransformation, #cybersecurity...
```

```
          tweet_metadata  tweet_retweet_count \
0  {'iso_language_code': 'en', 'result_type': 're...      0
1  {'iso_language_code': 'en', 'result_type': 're...      2
2  {'iso_language_code': 'en', 'result_type': 're...     39
3  {'iso_language_code': 'en', 'result_type': 're...      5
4  {'iso_language_code': 'en', 'result_type': 're...      0
```

```
          tweet_favorite_count  tweet_favorited  tweet_retweeted  tweet_lang
0              0              False              False          en
1              0              False              False          en
2              0              False              False          en
3              0              False              False          en
4              0              False              False          en
```

```
[972]: #View dataset info
TW.info
```

```
[972]: <bound method DataFrame.info of                                tweet_id
tweet_full_text \
0      1567629223795527681  As Arkansas' first Comprehensive Stroke Certif...
1      1567582846612553728  RT @AndreaGrammer: Work-life balance is at the...
2      1567582787070304256  RT @OntarioGreens: $10 billion can go a long w...
3      1567582767625428992  RT @modrnhealthcr: #NEW: Insurance companies...
4      1567582720460570625  ICYMI: Our recent blogs on Cybersecurity in Ac...
...
1169   1567583004209332227  RT @PotomacPhotonic: Potomac #Innovation Repor...
1170   1567582945342267393  Not a cent towards workers who would like to a...
1171   1567582936014241792  The @hfmaorg Region 9 presents "The Value of E...
1172   1567582892892782592  Happy physiotherapy day ..\n#bpt #physiothe...
1173   1567582846847533058  RT @KimcoStaffing: Apply now to work for #Medi...

                                tweet_entities \
0      {'hashtags': [{'text': 'Healthcare', 'indices'...
1      {'hashtags': [{'text': 'PremiseHealth', 'indic...
2      {'hashtags': [{'text': 'Healthcare', 'indices'...
3      {'hashtags': [{'text': 'NEW', 'indices': [20, ...
4      {'hashtags': [{'text': 'blogs', 'indices': [18...
...
1169   {'hashtags': [{'text': 'Innovation', 'indices'...
1170   {'hashtags': [{'text': 'SKPolli', 'indices': [2...
1171   {'hashtags': [{'text': 'webinar', 'indices': [...
1172   {'hashtags': [{'text': 'bpt', 'indices': [30, ...
1173   {'hashtags': [{'text': 'MediQuestStaffing', 'i...

                                tweet_entities_hashtags \
0
                                #Healthcare
1
                                #PremiseHealth, #hiring
2
                                #Healthcare
3
                                #NEW
4      #blogs, #digitaltransformation, #cybersecurity...
...
1169   #Innovation, #precisionFabrication, #Microfluidic
1170
                                #SKPolli, #healthcare
1171
                                #webinar, #ESG, #healthcare
1172   #bpt, #physiotherapy, #HealthyNation, #healthc...
1173   #MediQuestStaffing, #Hospital, #shift, #Newpor...

                                tweet_metadata  tweet_retweet_count \
0      {'iso_language_code': 'en', 'result_type': 're...      0
1      {'iso_language_code': 'en', 'result_type': 're...      2
2      {'iso_language_code': 'en', 'result_type': 're...      39
```

```

3      {'iso_language_code': 'en', 'result_type': 're...      5
4      {'iso_language_code': 'en', 'result_type': 're...      0
...
1169  {'iso_language_code': 'en', 'result_type': 're...      1
1170  {'iso_language_code': 'en', 'result_type': 're...      0
1171  {'iso_language_code': 'en', 'result_type': 're...      0
1172  {'iso_language_code': 'en', 'result_type': 're...      0
1173  {'iso_language_code': 'en', 'result_type': 're...      2

```

```

      tweet_favorite_count  tweet_favorited  tweet_retweeted  tweet_lang
0              0          False          False          en
1              0          False          False          en
2              0          False          False          en
3              0          False          False          en
4              0          False          False          en
...
1169              0          False          False          en
1170              1          False          False          en
1171              0          False          False          en
1172              5          False          False          en
1173              0          False          False          en

```

[1174 rows x 10 columns]>

```

[973]: #View column types
print(TW.dtypes)

```

```

tweet_id          int64
tweet_full_text    object
tweet_entities     object
tweet_entities_hashtags  object
tweet_metadata     object
tweet_retweet_count  int64
tweet_favorite_count  int64
tweet_favorited     bool
tweet_retweeted     bool
tweet_lang         object
dtype: object

```

```

[974]: #View if there are null values - only in the hashtags column
TW.isnull().sum()

```

```

[974]: tweet_id          0
      tweet_full_text    0
      tweet_entities     0
      tweet_entities_hashtags  167
      tweet_metadata     0

```

```

tweet_retweet_count      0
tweet_favorite_count      0
tweet_favorited          0
tweet_retweeted          0
tweet_lang               0
dtype: int64

```

```

[975]: #No duplicates found
print("Duplicates:",TW.duplicated().sum())

```

Duplicates: 0

```

[976]: #Rename columns to easier conventions
TW = TW.rename(columns={
    'tweet_id': 'ID',
    'tweet_full_text': 'Full',
    'tweet_entities': 'Entities',
    'tweet_entities_hashtags': 'Hashtags',
    'tweet_metadata': 'Meta',
    'tweet_retweet_count': '#_Retweet',
    'tweet_favorite_count': '#_Fav',
    'tweet_favorited': 'Fav_Bi',
    'tweet_retweeted': 'Retweet_Bi',
    'tweet_lang': 'Lang'
})

print(TW)

```

	ID	Full \
0	1567629223795527681	As Arkansas' first Comprehensive Stroke Certif...
1	1567582846612553728	RT @AndreaGrammer: Work-life balance is at the...
2	1567582787070304256	RT @OntarioGreens: \$10 billion can go a long w...
3	1567582767625428992	RT @modrnhealthcr: #NEW: Insurance companies...
4	1567582720460570625	ICYMI: Our recent blogs on Cybersecurity in Ac...
...	...	...
1169	1567583004209332227	RT @PotomacPhotonic: Potomac #Innovation Repor...
1170	1567582945342267393	Not a cent towards workers who would like to a...
1171	1567582936014241792	The @hfmaorg Region 9 presents "The Value of E...
1172	1567582892892782592	Happy physiotherapy day ..\n#bpt #physiothe...
1173	1567582846847533058	RT @KimcoStaffing: Apply now to work for #Medi...

	Entities \
0	{'hashtags': [{'text': 'Healthcare', 'indices'...
1	{'hashtags': [{'text': 'PremiseHealth', 'indic...
2	{'hashtags': [{'text': 'Healthcare', 'indices'...
3	{'hashtags': [{'text': 'NEW', 'indices': [20, ...
4	{'hashtags': [{'text': 'blogs', 'indices': [18...
...	...

```

1169 {'hashtags': [{'text': 'Innovation', 'indices'...
1170 {'hashtags': [{'text': 'SKPoli', 'indices': [2...
1171 {'hashtags': [{'text': 'webinar', 'indices': [...
1172 {'hashtags': [{'text': 'bpt', 'indices': [30, ...
1173 {'hashtags': [{'text': 'MediQuestStaffing', 'i...

```

```

                                Hashtags \
0                                #Healthcare
1                                #PremiseHealth, #hiring
2                                #Healthcare
3                                #NEW
4                                #blogs, #digitaltransformation, #cybersecurity...
...
1169 #Innovation, #precisionFabrication, #Microfluidic
1170                                #SKPoli, #healthcare
1171                                #webinar, #ESG, #healthcare
1172 #bpt, #physiotherapy, #HealthyNation, #healthc...
1173 #MediQuestStaffing, #Hospital, #shift, #Newpor...

```

```

                                Meta  #_Retweet  #_Fav  \
0      {'iso_language_code': 'en', 'result_type': 're...      0      0
1      {'iso_language_code': 'en', 'result_type': 're...      2      0
2      {'iso_language_code': 'en', 'result_type': 're...     39      0
3      {'iso_language_code': 'en', 'result_type': 're...      5      0
4      {'iso_language_code': 'en', 'result_type': 're...      0      0
...
1169 {'iso_language_code': 'en', 'result_type': 're...      1      0
1170 {'iso_language_code': 'en', 'result_type': 're...      0      1
1171 {'iso_language_code': 'en', 'result_type': 're...      0      0
1172 {'iso_language_code': 'en', 'result_type': 're...      0      5
1173 {'iso_language_code': 'en', 'result_type': 're...      2      0

```

```

Fav_Bi  Retweet_Bi  Lang
0      False      False  en
1      False      False  en
2      False      False  en
3      False      False  en
4      False      False  en
...
1169  False      False  en
1170  False      False  en
1171  False      False  en
1172  False      False  en
1173  False      False  en

```

[1174 rows x 10 columns]

```
[977]: #Make sure column names have changed  
print(TW.columns)
```

```
Index(['ID', 'Full', 'Entities', 'Hashtags', 'Meta', '#_Retweet', '#_Fav',  
      'Fav_Bi', 'Retweet_Bi', 'Lang'],  
      dtype='object')
```

```
[978]: #Unique tweets which have been retweeted  
TW["#_Retweet"].nunique()
```

```
[978]: 47
```

```
[979]: #Over 10,000 total retweets  
TW["#_Retweet"].sum()
```

```
[979]: 10131
```

```
[980]: #View top 20 retweets - skewed towards top - particualrly top 3 tweets  
TW["#_Retweet"].value_counts().head(20)
```

```
[980]: #_Retweet  
0      526  
1      215  
2      114  
3       70  
5       35  
4       27  
7       18  
12      16  
8       15  
73      14  
9       13  
6       12  
208     12  
35      10  
37       6  
11       6  
10       5  
53       5  
44       4  
150      4  
Name: count, dtype: int64
```

```
[981]: #Unique tweets which have been favorited  
TW["#_Fav"].nunique()
```

```
[981]: 20
```

```
[982]: #Over 400 total favorites
TW["#_Fav"].sum()
```

```
[982]: 436
```

```
[983]: #View top 20 favorites - skewed towards top - particualrly top 3-4 tweets
TW["#_Fav"].value_counts().head(20)
```

```
[983]: #_Fav
0      1027
1        91
2        16
3        13
4         7
5         5
6         2
17        1
12        1
10        1
8         1
13        1
11        1
7         1
20        1
28        1
14        1
18        1
9         1
42        1
Name: count, dtype: int64
```

```
[984]: #Use groupby to view as new df with tweet text + favorite count + percentage of
↳all favorites
Fav = (
    TW.groupby("Full")["#_Fav"]
      .sum()
      .reset_index()
)

Fav["Pct"] = Fav["#_Fav"] / Fav["#_Fav"].sum() * 100

Fav.sort_values("Pct", ascending=False).head(20)
```

```
[984]:
```

	Full	#_Fav	Pct
353	Lipid-Lowering Drugs\n\n#TipsForNewDocs #MedEd...	42	10
944	You ready for \$JCO @_JennyCo \n\n#Healthcare...	28	6
281	How health insurance works \n\n#comedy #adul...	20	5

413	Our nat'l choices re: #healthcare systems aren...	18	4
265	Heart Failure, Myocardial Infarction & imm...	17	4
370	More data that our 13+ #healthcare systems ...	14	3
357	Looking forward to speaking at #ConV2X on Sep ...	13	3
70	@CapricornFMNews We have waiting to hear this ...	12	3
771	September is #WomenInMedicine Month! Thrilled ...	11	3
794	Thanks & Happy #WomenInMedicine Month to A...	10	2
908	We're proud to partner with Drs. Michael P. Ko...	9	2
50	.@MedtechWomen's annual conference brings lead...	8	2
45	...if you don't want to #ProtectWomen, have a ...	7	2
289	I had a great time at yesterday's Pancho Medra...	6	1
49	.@KateWalshCEO has been nominated as one of @m...	6	1
424	Pregnant? Please don't kill the tiny person gr...	5	1
255	Happy physiotherapy day ..\n#bpt #physiothe...	5	1
290	I have some really BIG news! After months of p...	5	1
6	#AI tools in #healthcare!\n\nInfographic by @i...	5	1
3	"To show my support for the @SHNcares ER docto...	5	1

[985]: *#Use groupby to view as new df with tweet text + retweet count + percentage of*
↳ all favorites

```

Rtweet = (
    Tw.groupby("Full")["#_Retweet"]
        .sum()
        .reset_index()
)

Rtweet["Pct"] = Rtweet["#_Retweet"] / Rtweet["#_Retweet"].sum() * 100

Rtweet.sort_values("Pct", ascending=False).head(20)

```

[985]:

	Full	#_Retweet	Pct
695	RT @khaleejtimes: .@BurjeelHoldings, a private...	3117	31
721	RT @ricochet_en: Health care in Canada is coll...	801	8
525	RT @Khulood_Almani: #Applications of #AI in #...	600	6
671	RT @healthcare_NFT_: Really excited to be work...	365	4
458	RT @CURE_Ecosystem: EXCLUSIVE footage. Just fo...	315	3
603	RT @UltimaLionsDen: Temitope is looking to boo...	303	3
517	RT @JeffWaltersSask: A good read on what nurse...	257	3
585	RT @SoniaFurstenau: Germany has one of the top...	255	3
672	RT @healthcare_NFT_: The #HealthCare Heroes jo...	252	2
441	RT @AWScloudIndia: How is #data revolutionizin...	228	2
572	RT @Ronald_vanLoon: A global team of scientist...	212	2
670	RT @healthcare_NFT_: In addition to announceme...	204	2
675	RT @holisticchamber: Even Time Magazine has so...	186	2
573	RT @Ronald_vanLoon: Rehabilitation #Robotic Gl...	176	2
682	RT @imedverse: I.V Drug Calculations Cheat She...	169	2
526	RT @Khulood_Almani: This #wearable Chair from...	156	2

683	RT @imedverse: Lipid-Lowering Drugs\n\n#TipsFo...	120	1
524	RT @Khulood_Almani: #Healthcare #DigitalTransf...	107	1
523	RT @KatharineSmart: We need more people on our...	82	1
587	RT @SoniaFurstenau: People in BC are dying of ...	81	1

```
[986]: #Create new df
tweets_text = TW[['Full']]
```

```
[987]: #Make sure new df contains expected amount of content
tweets_text["Full"].nunique()
```

```
[987]: 961
```

```
[988]: #View first 5 full tweet text
tweets_text.head
```

```
[988]: <bound method NDFrame.head of
Full
0    As Arkansas' first Comprehensive Stroke Certif...
1    RT @AndreaGrammer: Work-life balance is at the...
2    RT @OntarioGreens: $10 billion can go a long w...
3    RT @modrnhealthcr: #NEW: Insurance companies...
4    ICYMI: Our recent blogs on Cybersecurity in Ac...
...
1169 RT @PotomacPhotonic: Potomac #Innovation Repor...
1170 Not a cent towards workers who would like to a...
1171 The @hfmaorg Region 9 presents "The Value of E...
1172 Happy physiotherapy day ..\n#bpt #physiothe...
1173 RT @KimcoStaffing: Apply now to work for #Medi...

[1174 rows x 1 columns]>
```

```
[989]: #Create empty list
tags = []

#Create loop for extraction of tags
for y in [x.split(' ') for x in tweets_text['Full'].values]:
    for z in y:
        if '#' in z:
            # Change to lowercase.
            tags.append(z.lower())
```

```
[990]: #Create new df for tags
data = pd.Series(tags).value_counts().reset_index()
```

```
[991]: #Assign columns to df
data.columns = ["Hashtag", "Count"]
```

```
[992]: #View df types
print(data.dtypes)
```

```
Hashtag    object
Count      int64
dtype: object
```

```
[993]: #Count hashtags
data = (
    pd.Series(tags)
      .value_counts()
      .reset_index()
      .rename(columns={"index": "Hashtag", 0: "Count"})
)

#View top 5 hashtags
data.head()
```

```
[993]:
```

	Hashtag	count
0	#healthcare	716
1	#health	80
2	#medicine	41
3	#ai	40
4	#job	38

```
[994]: #View bottom 5 hashtags
data.tail()
```

```
[994]:
```

	Hashtag	count
1749	#evestudy	1
1750	#patientdata...	1
1751	#secure	1
1752	#sms	1
1753	\n#csjmu	1

```
[995]: #Cut down the representation to Top 20 hashtags by count
top20 = data.sort_values("count", ascending=False).head(20)

#Set plot metrics
plt.figure(figsize=(14, 6))

ax = sns.barplot(
    data=top20,
    x="Hashtag",
    y="count",
    order=top20["Hashtag"]
)
```

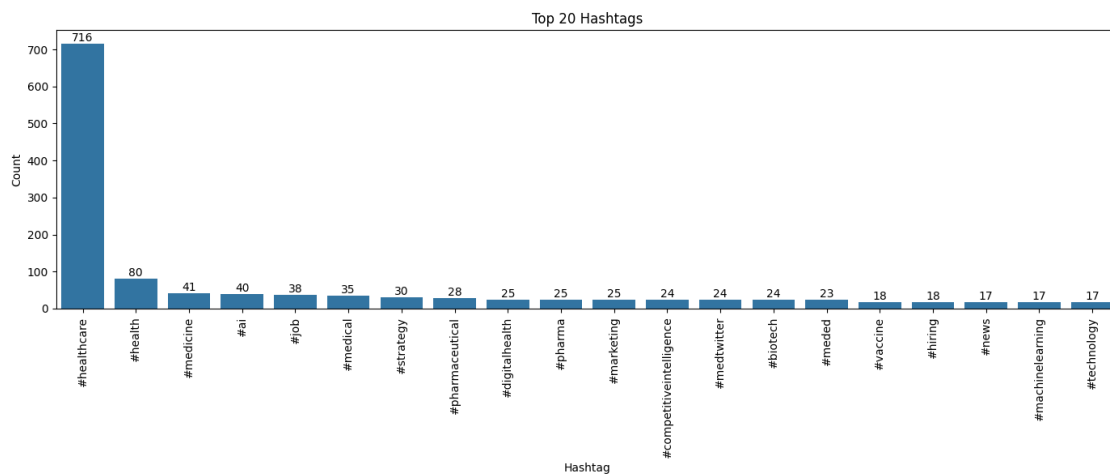
```

#Add value labels above bars
for p in ax.patches:
    ax.annotate(
        f"{int(p.get_height()):,}",
        (p.get_x() + p.get_width() / 2, p.get_height()),
        ha="center",
        va="bottom",
        fontsize=10
    )

plt.xticks(rotation=90)
plt.title("Top 20 Hashtags")
plt.xlabel("Hashtag")
plt.ylabel("Count")

plt.tight_layout()
plt.show()

```



[996] : #a. how your team can utilise tweets to provide feedback to the stakeholders
 #Evidently from the high hashtag count of "Healthcare" Twitter can be offered
 ↳ as a tool at the disposal of the NHS...
 #and people are interested in innovation (AI/data-driven) approaches giving the
 ↳ NHS an opportunity to cease given
 #the online interest in healthcare operations, functions, and innovations

#b. whether the tweets add value to the overall project
 #I find that they add value because they provide a look at what the overall
 ↳ population/patients think about healthcare,
 #what their interests are and what their complaints are

#c. how the NHS can utilise tweets

#Modern implementations can be advertised to patients - furthermore the NHS is

→seldomly mentioned offering an invaluable

#opportunity to control the narrative of the British healthcare system and the

→use of a data-driven approach to improve

#Patient/GP coordination

#d. any suggestions regarding data collection or analysis to improve future

→efforts

#It would be helpful to scrape tweets which specifically reference the NHS...

#This would provide more pointed commentary and insights about what the UK

→population thinks about their healthcare system